TỔNG CÔNG TY BƯU ĐIỆN VIỆT NAM

TRUNG TÂM ĐỔI MỚI SÁNG TẠO – PHÒNG AI

Triển khai mô hình AI trong lĩnh vực Chatbot/Trợ lý ảo, ứng dụng các kỹ thuật RAG hoặc fine-tuning mô hình ngôn ngữ lớn có khả năng trả lời thông minh, chính xác, ngữ cảnh hóa câu trả lời dựa trên tài liệu, dữ liệu nội bộ hoặc thông tin chuyên ngành của Tổng công ty.

BÁO CÁO KỸ THUẬT – ĐỀ XUẤT TRIỂN KHAI

Kính gửi:	Ban lãnh đạo Tổng công ty Bưu điện Việt Nam
Đơn vị thực hiện:	Phòng AI, Trung tâm Đổi mới Sáng tạo
Ngày báo cáo:	[NGÀY/THÁNG/NĂM]

[NĂM]

Lời nói đầu

Trong bối cảnh chuyển đổi số của ngành bưu chính, việc ứng dụng trí tuệ nhân tạo vào công tác tra cứu, hỏi đáp và hỗ trợ nghiệp vụ là một hướng đi có tính thực tiễn cao. Báo cáo này trình bày kết quả nghiên cứu và đề xuất triển khai một trợ lý ảo thông minh dựa trên mô hình ngôn ngữ lớn (LLM) kết hợp kỹ thuật Retrieval-Augmented Generation (RAG) và fine-tuning, phục vụ nhu cầu trả lời chính xác, ngữ cảnh hóa dựa trên tài liệu nội bộ của Tổng công ty Bưu điện Việt Nam (VNPost).

Báo cáo được xây dựng trên cơ sở khảo sát hệ thống và dữ liệu thực tế đang vận hành, kết hợp phân tích lý thuyết và kết quả thực nghiệm. Các nội dung chính gồm: cơ sở lý thuyết về RAG và fine-tuning; kiến trúc hệ thống đa tác tử (multi-agent) đề xuất; kết quả triển khai thực nghiệm; đánh giá định lượng và định tính; cùng lộ trình và khuyến nghị triển khai.

Phòng AI xin gửi lời cảm ơn tới Ban lãnh đạo và các đơn vị liên quan đã tạo điều kiện, chia sẻ bối cảnh nghiệp vụ và dữ liệu để hoàn thành báo cáo này.

Phòng AI, Trung tâm Đổi mới Sáng tạo

Tóm tắt điều hành

Trong bối cảnh chuyển đổi số của ngành bưu chính, Tổng công ty Bưu điện Việt Nam (VNPost) sở hữu một khối lượng lớn tài liệu nghiệp vụ nội bộ: quy trình, quyết định, biểu phí dịch vụ, quy định về hàng hóa và chính sách bồi thường, cùng các nghiệp vụ tra cứu bưu gửi và bưu cục. Việc tra cứu thủ công tốn nhiều thời gian, trong khi các kênh hỏi đáp truyền thống khó đáp ứng nhu cầu tra cứu tức thời, chính xác và có trích nguồn. Báo cáo này nghiên cứu việc triển khai một trợ lý ảo thông minh dựa trên mô hình ngôn ngữ lớn (LLM) kết hợp kỹ thuật Retrieval-Augmented Generation (RAG) và fine-tuning, hướng tới câu trả lời chính xác, ngữ cảnh hóa và bám sát tài liệu nội bộ.

Về phương pháp, báo cáo khảo sát cơ sở lý thuyết về LLM, RAG và fine-tuning; thiết kế kiến trúc trợ lý ảo đa tác tử (multi-agent) gồm một agent điều phối và các agent chuyên biệt, giao tiếp qua các giao thức MCP và A2A; triển khai pipeline RAG lai (hybrid) kết hợp sparse retrieval BM25 và dense retrieval trên vector database Qdrant, kèm reranker; đồng thời đề xuất pipeline fine-tuning cho cả ba loại mô hình: LLM (LoRA/QLoRA), embedding model và reranker. Toàn bộ mô hình được phục vụ cục bộ (on-premise) bằng vLLM để bảo đảm dữ liệu nội bộ không rời khỏi hạ tầng của VNPost.

Kết quả thực nghiệm trên tập tài liệu nội bộ (bộ mẫu 6 câu hỏi có đáp án tham chiếu) cho thấy dense retrieval đạt Recall@20 = 100%, trong khi cấu hình sản xuất sau rerank đạt Recall@5 = 50% và MRR = 0,3149; phân tích lỗi cho thấy nguyên nhân chính nằm ở khâu rerank chứ không phải khâu retrieval. Với miền EMS, bộ đánh giá gồm 99 câu hỏi đã được thu thập phản hồi đầy đủ. Báo cáo cũng trình bày kết quả đánh giá sinh văn bản bằng LLM-as-judge, bộ đánh giá RAGAS đã xây dựng cho bộ 99 câu hỏi EMS, kết quả đo throughput của vLLM, ước tính hiệu quả kinh tế – xã hội, và đề xuất kiến trúc hybrid RAG + fine-tuning cùng lộ trình phát triển. Kết quả cho thấy RAG là giải pháp phù hợp và khả thi cho tri thức động, còn fine-tuning bổ trợ cho phong cách và nghiệp vụ ổn định, và sự kết hợp của hai hướng tiếp cận mang lại giá trị thực tiễn cao cho Tổng công ty Bưu điện Việt Nam.

Từ khóa: trợ lý ảo, chatbot, LLM, RAG, fine-tuning, LoRA, hybrid retrieval,

reranker, multi-agent, VNPost.

Danh mục từ viết tắt

Từ viết tắt	Diễn giải
A2A	Agent-to-Agent protocol – giao thức giao tiếp giữa các agent
API	Application Programming Interface – giao diện lập trình ứng dụng
BM25	Best Matching 25 – hàm xếp hạng sparse retrieval dựa trên xác suất
CoT	Chain-of-Thought – chuỗi suy luận trong prompt
CSKH	Chăm sóc khách hàng
DPO	Direct Preference Optimization – tối ưu hóa theo sở thích trực tiếp
DPR	Dense Passage Retrieval – dense retrieval đoạn văn
EMS	Express Mail Service – dịch vụ chuyển phát nhanh
FP8	Định dạng số thực dấu phẩy động 8 bit
GPU	Graphics Processing Unit – đơn vị xử lý đồ họa
K8s	Kubernetes – nền tảng điều phối container
LLM	Large Language Model – mô hình ngôn ngữ lớn
LoRA	Low-Rank Adaptation – thích ứng hạng thấp
MCP	Model Context Protocol – giao thức ngữ cảnh mô hình
MLflow	Nền tảng quản lý vòng đời học máy
MRR	Mean Reciprocal Rank – nghịch đảo thứ hạng trung bình
nDCG	normalized Discounted Cumulative Gain
OCR	Optical Character Recognition – nhận dạng ký tự quang học
OTel	OpenTelemetry – chuẩn quan sát hệ thống
PEFT	Parameter-Efficient Fine-Tuning – fine-tuning hiệu quả tham số
PII	Personally Identifiable Information – thông tin định danh cá nhân

Từ viết tắt	Diễn giải
QLoRA	Quantized LoRA – LoRA trên mô hình lượng tử hóa (quantization)
Qdrant	Vector database mã nguồn mở
RAG	Retrieval-Augmented Generation – sinh tăng cường truy hồi
RAGAS	Framework đánh giá RAG tự động
RBAC	Role-Based Access Control – kiểm soát truy cập theo vai trò
RLHF	Reinforcement Learning from Human Feedback
SFT	Supervised Fine-Tuning – fine-tuning có giám sát
SSO	Single Sign-On – đăng nhập một lần
VNPost	Tổng công ty Bưu điện Việt Nam
vLLM	Thư viện phục vụ suy luận LLM hiệu năng cao

Mục lục

Lời nói đầu **1**

Tóm tắt điều hành **2**

Danh mục từ viết tắt **4**

1 Giới thiệu **12**
1.1 Bối cảnh và lý do chọn đề tài 12
1.2 Mục tiêu nghiên cứu 13
1.3 Đối tượng và phạm vi nghiên cứu 14
1.3.1 Đối tượng nghiên cứu 14
1.3.2 Phạm vi nghiên cứu 14
1.4 Phương pháp nghiên cứu 14
1.5 Đóng góp của đề tài 15
1.6 Cấu trúc báo cáo 16

2 Cơ sở lý thuyết và tổng quan **17**
2.1 Chatbot và trợ lý ảo 17
2.1.1 Khái niệm và phân loại 17
2.1.2 Trợ lý ảo và kiến trúc hội thoại 17
2.2 Mô hình ngôn ngữ lớn 18
2.2.1 Kiến trúc Transformer 18
2.2.2 Tokenization và biểu diễn từ 18
2.2.3 Quá trình huấn luyện 19
2.2.4 Sinh văn bản và prompt 19
2.3 RAG (Retrieval-Augmented Generation) 20
2.3.1 Khái niệm và động lực 20
2.3.2 Kiến trúc tổng quát 20
2.3.3 Chunking và tiền xử lý tài liệu 21
2.3.4 Embedding vector và vector database 21
2.3.5 Sparse retrieval, dense retrieval và hybrid retrieval . . . 21
2.3.6 Rerank 22
2.3.7 Các biến thể của RAG 22
2.4 Fine-tuning mô hình ngôn ngữ lớn (Fine-tuning) 23

2.4.1 Khái niệm và các mức độ . . . 23
2.4.2 Fine-tuning có giám sát và căn chỉnh . . . 23
2.4.3 LoRA . . . 23
2.4.4 QLoRA . . . 24
2.4.5 Khi nào nên dùng fine-tuning . . . 24
2.5 So sánh RAG và fine-tuning, kiến trúc hybrid . . . 24
2.6 Các thước đo đánh giá . . . 25
2.6.1 Thước đo retrieval . . . 25
2.6.2 Thước đo sinh văn bản . . . 26
2.7 Nền tảng triển khai . . . 26
2.7.1 Phục vụ inference với vLLM . . . 26
2.7.2 Quantization FP8 . . . 26
2.7.3 Giao thức MCP và A2A . . . 26
2.7.4 Quan sát hệ thống và quản lý prompt . . . 27
Kết chương . . . 27

3 Kiến trúc hệ thống đề xuất 28
3.1 Đặc thù bài toán tại VNPost . . . 28
3.1.1 Đặc điểm dữ liệu nội bộ . . . 28
3.1.2 Các nhóm nghiệp vụ . . . 29
3.2 Yêu cầu chức năng và phi chức năng . . . 29
3.2.1 Yêu cầu chức năng . . . 29
3.2.2 Yêu cầu phi chức năng . . . 29
3.3 Kiến trúc tổng thể multi-agent . . . 29
3.4 Pipeline RAG chi tiết . . . 30
3.4.1 Pha indexing . . . 30
3.4.2 Pha retrieval và sinh . . . 31
3.4.3 Tham số hóa và cấu hình . . . 31
3.5 Pipeline fine-tuning đề xuất . . . 31
3.5.1 Thu thập và chuẩn bị dữ liệu . . . 33
3.5.2 Fine-tuning LLM (LoRA/QLoRA) . . . 33
3.5.3 Fine-tuning embedding model . . . 33
3.5.4 Fine-tuning reranker . . . 34
3.6 Kiến trúc hybrid RAG + Fine-tuning . . . 35
3.7 Hạ tầng và bảo mật . . . 36
3.7.1 Hạ tầng phục vụ . . . 36
3.7.2 Bảo mật dữ liệu . . . 36

Kết chương 36

4 Triển khai thực nghiệm 37
4.1 Môi trường và công cụ 37
4.2 Dữ liệu 38
4.2.1 Các kho tài liệu 38
4.2.2 Bộ đánh giá 38
4.2.3 Tiền xử lý 38
4.3 Cài đặt RAG 39
4.3.1 Chunking và embedding 39
4.3.2 Cấu hình retrieval 39
4.3.3 Embedding câu hỏi và prompt 39
4.4 Thiết kế fine-tuning 40
4.5 Triển khai multi-agent 41
4.5.1 Agent điều phối 41
4.5.2 Các agent chuyên biệt và công cụ 41
4.5.3 Quy tắc định tuyến 42
4.6 Hệ thống quan sát và quản lý prompt 42
4.7 Khó khăn và giải pháp 42
Kết chương 42

5 Kết quả và đánh giá 44
5.1 Phương pháp đánh giá 44
5.2 Kết quả retrieval 45
5.2.1 Kết quả tổng hợp 45
5.2.2 Phân tích lỗi 46
5.3 Kết quả sinh văn bản và đánh giá agent 46
5.3.1 Đánh giá bởi LLM-as-judge 46
5.3.2 Bộ đánh giá RAGAS trên bộ 99 câu hỏi EMS 46
5.4 So sánh RAG, fine-tuning và kiến trúc hybrid 47
5.5 Hiệu năng hệ thống 47
5.6 Case study định tính 49
5.7 Hiệu quả kinh tế – xã hội 49
5.8 Thảo luận 50
5.8.1 Ưu điểm 50
5.8.2 Hạn chế 50
5.8.3 Tính khái quát 51
Kết chương 51

6 Kết luận và hướng phát triển **52**
6.1 Kết luận 52
6.2 Hạn chế 53
6.3 Hướng phát triển 53
Kết chương 54

A Prompt hệ thống mẫu **55**
A.1. Prompt agent điều phối 55
A.2. Prompt agent RAG 55

B Bảng cấu hình tham số **57**

C Danh mục mô hình và thành phần **58**

D Mẫu hội thoại minh họa **59**

Danh sách hình vẽ

2.1 Sơ đồ khối một khối Transformer (một lớp) với self-attention và feed-forward. 19
2.2 Kiến trúc tổng quát của một hệ thống RAG. 20
2.3 Pipeline indexing tài liệu trong hệ thống RAG. 21
2.4 Pipeline hybrid retrieval kết hợp BM25, dense retrieval và rerank. 22
2.5 Minh họa LoRA: giữ nguyên W_0 và thêm cập nhật hạng thấp BA. 24

3.1 Kiến trúc multi-agent: host điều phối các agent chuyên biệt qua A2A. 30
3.2 Pipeline RAG đầy đủ từ indexing đến sinh câu trả lời. 32
3.3 Pipeline fine-tuning đề xuất cho ba loại mô hình: LLM, embedding và reranker. 34
3.4 Kiến trúc hybrid: router chọn RAG, mô hình fine-tuning hoặc kết hợp. 35

5.1 So sánh Recall giữa các cấu hình retrieval. 45
5.2 Throughput phục vụ lô của các cấu hình vLLM. 48

Danh sách bảng

2.1 So sánh RAG và fine-tuning theo các tiêu chí triển khai. 25

4.1 Các công cụ và thành phần chính của hệ thống. 37
4.2 Thống kê các kho tài liệu sử dụng trong thực nghiệm. 38
4.3 Cấu hình tham số retrieval RAG. 39
4.4 Cấu hình LoRA/QLoRA đề xuất cho fine-tuning LLM. 40
4.5 Cấu hình fine-tuning embedding model đề xuất. 40
4.6 Cấu hình fine-tuning reranker đề xuất. 41
4.7 Khó khăn gặp phải và giải pháp áp dụng. 43

5.1 Kết quả retrieval trên bộ mẫu (6 câu hỏi). 45
5.2 So sánh RAG, fine-tuning và kiến trúc hybrid trong bối cảnh VNPost. 47
5.3 Kết quả đo inference throughput vLLM. 48
5.4 Ước lượng hiệu quả kinh tế của giải pháp. 50

B.1 Cấu hình tham số tổng hợp của hệ thống. 57

C.1 Các mô hình và thành phần chính được sử dụng. 58

Chương 1

Giới thiệu

1.1 Bối cảnh và lý do chọn đề tài

Trong những năm gần đây, chuyển đổi số đã trở thành một trong những định hướng chiến lược trọng tâm của ngành bưu chính – viễn thông Việt Nam. Tổng công ty Bưu điện Việt Nam (Tổng công ty Bưu điện Việt Nam, sau đây gọi tắt là VNPost) vận hành một mạng lưới rộng khắp với hàng nghìn điểm phục vụ, hàng chục nghìn lao động và một danh mục dịch vụ đa dạng: bưu chính, chuyển phát nhanh (EMS), logistics, tài chính bưu chính và các dịch vụ công. Song song với hoạt động vận hành, VNPost sở hữu một khối lượng tài liệu nghiệp vụ nội bộ rất lớn, bao gồm quy trình nghiệp vụ, quyết định, công văn hướng dẫn, biểu phí dịch vụ, quy định về hàng hóa cấm gửi, chính sách bồi thường và các tiêu chuẩn chất lượng phát.

Thực tế vận hành cho thấy việc tra cứu thủ công khối tài liệu này tiêu tốn đáng kể thời gian của cán bộ công nhân viên. Khi một khách hàng hỏi về cước phí EMS, thời gian toàn trình, quy định về hàng hóa đặc biệt hay chính sách bồi thường, nhân viên phải mở nhiều văn bản khác nhau, tìm đúng điều khoản, rồi tổng hợp câu trả lời. Các kênh hỏi đáp truyền thống như danh mục câu hỏi thường gặp (FAQ) hoặc tổng đài chỉ có thể đáp ứng một phần nhu cầu: FAQ cứng nhắc và nhanh lỗi thời, còn tổng đài phụ thuộc vào số lượng nhân viên và thời gian làm việc. Hệ quả là trải nghiệm khách hàng bị ảnh hưởng và chi phí vận hành tăng lên.

Sự phát triển của các LLM (Large Language Model) mở ra một hướng giải quyết mới. Với khả năng hiểu ngôn ngữ tự nhiên và sinh văn bản mạch lạc, LLM có thể đóng vai trò lõi của một trợ lý ảo có khả năng hội thoại tự nhiên. Tuy nhiên, bản thân một LLM tổng quát không thể trả lời chính xác về tài liệu nội bộ của một tổ chức cụ thể: nó không được huấn luyện trên dữ liệu đó, dễ sinh thông tin sai lệch (hiện tượng “hallucination”), và không thể trích dẫn nguồn để người dùng kiểm chứng. Hai hướng tiếp cận chính được cộng đồng nghiên cứu và công nghiệp áp dụng để khắc phục hạn chế này là:

- **RAG (Retrieval-Augmented Generation – RAG):** tách tri thức ra khỏi tham số mô hình, lấy các document chunk liên quan từ một kho tri thức bên ngoài tại thời điểm inference, rồi đưa chúng vào ngữ cảnh để mô hình sinh câu trả lời có căn cứ và có thể trích nguồn.
- **Fine-tuning (fine-tuning):** tiếp tục huấn luyện mô hình trên dữ liệu chuyên ngành để điều chỉnh hành vi, phong cách và kiến thức miền, giúp mô hình trả lời đúng "giọng" nghiệp vụ và định dạng mong muốn.

Đối với một tổ chức như VNPost, bài toán đặt ra không chỉ là kỹ thuật mà còn là bảo mật dữ liệu: tài liệu nghiệp vụ nội bộ và thông tin khách hàng không được phép rời khỏi hạ tầng của Tổng công ty. Điều này dẫn tới yêu cầu triển khai mô hình cục bộ (on-premise) thay vì gọi các dịch vụ đám mây bên ngoài. Báo cáo này xuất phát từ chính những yêu cầu thực tiễn đó: nghiên cứu và triển khai một trợ lý ảo thông minh cho VNPost, kết hợp RAG và fine-tuning, có khả năng trả lời chính xác, ngữ cảnh hóa dựa trên tài liệu nội bộ, vận hành an toàn trên hạ tầng riêng và có thể mở rộng theo nhu cầu nghiệp vụ.

1.2 Mục tiêu nghiên cứu

Đề tài hướng tới các mục tiêu sau:

1. Hệ thống hóa cơ sở lý thuyết về chatbot/trợ lý ảo, mô hình ngôn ngữ lớn, kỹ thuật RAG và kỹ thuật fine-tuning, làm rõ ưu nhược điểm và phạm vi áp dụng của từng hướng tiếp cận.
2. Thiết kế một kiến trúc trợ lý ảo multi-agent phù hợp với đặc thù dữ liệu và nghiệp vụ của VNPost, trong đó các agent chuyên biệt đảm nhận các miền nghiệp vụ khác nhau và giao tiếp qua các giao thức chuẩn hóa.
3. Triển khai và thực nghiệm một pipeline RAG lai kết hợp sparse retrieval BM25 và dense retrieval trên vector database, có bước rerank, nhằm tạo câu trả lời bám sát tài liệu.
4. Đề xuất và mô tả pipeline fine-tuning cho cả ba loại mô hình – LLM (LoRA/QLoRA), embedding model và reranker – cùng tiêu chí quyết định khi nào nên dùng RAG, khi nào nên dùng mô hình đã fine-tuning, và cách kết hợp hai hướng tiếp cận trong một kiến trúc hybrid.
5. Đánh giá hệ thống bằng các thước đo định lượng (Recall@k, MRR, các thước đo RAGAS, đánh giá bởi mô hình ngôn ngữ) và định tính (case

study hội thoại), từ đó rút ra bài học triển khai và đề xuất lộ trình phát triển.

1.3 Đối tượng và phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là các phương pháp và kiến trúc triển khai trợ lý ảo dựa trên mô hình ngôn ngữ lớn cho miền nghiệp vụ bưu chính, cụ thể là kỹ thuật RAG và kỹ thuật fine-tuning, cùng các thành phần hỗ trợ như embedding, vector database, rerank và hạ tầng phục vụ mô hình.

1.3.2 Phạm vi nghiên cứu

Đề tài tập trung vào miền dữ liệu và nghiệp vụ của VNPost, bao gồm:

- **Tài liệu văn thư và quy trình nội bộ:** các quyết định, công văn, quy trình nghiệp vụ (tập thử nghiệm gồm 17 tài liệu văn thư đại diện).
- **Tài liệu sản phẩm – dịch vụ:** biểu phí, quy định hàng hóa cấm gửi, chính sách bồi thường, thời gian toàn trình (tập thử nghiệm gồm 119 tài liệu).
- **Tài liệu dịch vụ chuyển phát nhanh EMS:** các quyết định biểu phí, quy định hàng hóa đặc biệt, chính sách bồi thường EMS (khoảng 30 tài liệu).
- **Nghiệp vụ tra cứu:** tra cứu tình trạng bưu gửi, tra cứu thông tin bưu cục, và tra cứu chất lượng phát.
- **Bộ đánh giá:** bộ gồm 99 cặp câu hỏi – đáp án tham chiếu cho miền EMS.

Về mặt kỹ thuật, đề tài giới hạn ở các mô hình mã nguồn mở phục vụ cục bộ, không sử dụng các mô hình thương mại qua API đám mây, nhằm bảo đảm yêu cầu bảo mật dữ liệu nội bộ.

1.4 Phương pháp nghiên cứu

Đề tài sử dụng kết hợp nhiều phương pháp:

1. **Nghiên cứu tài liệu:** khảo sát các công trình khoa học và tài liệu kỹ thuật về Transformer, RAG, fine-tuning, LoRA/QLoRA, các thước đo đánh giá và hạ tầng phục vụ inference, nhằm xây dựng nền tảng lý thuyết và lựa chọn công nghệ.
2. **Phân tích và thiết kế hệ thống:** phân tích yêu cầu nghiệp vụ của VNPost, từ đó thiết kế kiến trúc multi-agent, pipeline RAG và pipeline fine-tuning cùng kiến trúc hybrid.
3. **Thực nghiệm:** triển khai hệ thống thử nghiệm, xây dựng các pipeline xử lý dữ liệu – từ nhận dạng ký tự quang học (OCR), chuẩn hóa tài liệu, chunking, embedding vector, đến hybrid retrieval và rerank; tiến hành đo đạc trên dữ liệu nội bộ.
4. **Đánh giá định lượng:** sử dụng các thước đo retrieval (Recall@k, MRR), các thước đo sinh văn bản (bộ thước đo RAGAS) và đánh giá bởi mô hình ngôn ngữ (LLM-as-judge); đo hiệu năng inference của hệ thống phục vụ.
5. **Đánh giá định tính:** phân tích các tình huống hội thoại thực tế, đánh giá mức độ hữu ích của câu trả lời và tính khả thi khi triển khai ở quy mô lớn.
6. **Ước lượng hiệu quả:** ước tính lợi ích kinh tế – xã hội dựa trên thời gian tiết kiệm được của người dùng so với quy trình thủ công.

1.5 Đóng góp của đề tài

Các đóng góp chính của đề tài bao gồm:

- **Về lý thuyết:** cung cấp một bức tranh tổng hợp, có hệ thống về RAG và fine-tuning trong bối cảnh triển khai trợ lý ảo doanh nghiệp, đặc biệt nhấn mạnh sự đánh đổi giữa tri thức động và hành vi miền, làm cơ sở cho kiến trúc hybrid.
- **Về kiến trúc:** đề xuất một kiến trúc trợ lý ảo multi-agent kết hợp RAG lai và fine-tuning cho miền bưu chính, có khả năng mở rộng và bảo đảm dữ liệu nội bộ không rời khỏi hạ tầng tổ chức.
- **Về thực nghiệm:** triển khai và đo đạc trên dữ liệu và nghiệp vụ thực tế của VNPost, cung cấp bằng chứng định lượng về hiệu quả của từng thành phần (sparse retrieval, dense retrieval, rerank) và chỉ ra điểm nghẽn cần cải thiện.

- **Về đánh giá:** xây dựng quy trình đánh giá kết hợp thước đo retrieval, thước đo RAGAS và đánh giá bởi LLM, phù hợp với đặc thù tiếng Việt và miền nghiệp vụ hẹp.
- **Về thực tiễn:** đề xuất lộ trình triển khai và ước lượng hiệu quả kinh tế, giúp tổ chức ra quyết định đầu tư và mở rộng hệ thống.

1.6 Cấu trúc báo cáo

Ngoài phần mở đầu và phụ lục, báo cáo được tổ chức thành sáu chương:

Chương 1 – Giới thiệu: trình bày bối cảnh, mục tiêu, đối tượng, phạm vi, phương pháp nghiên cứu, đóng góp và cấu trúc báo cáo.

Chương 2 – Cơ sở lý thuyết và tổng quan: hệ thống hóa kiến thức về chatbot, mô hình ngôn ngữ lớn, RAG, fine-tuning, kiến trúc hybrid, các thước đo đánh giá và nền tảng triển khai.

Chương 3 – Kiến trúc hệ thống đề xuất: phân tích bài toán, yêu cầu và thiết kế kiến trúc multi-agent, pipeline RAG, pipeline fine-tuning và kiến trúc hybrid cho VNPost.

Chương 4 – Triển khai thực nghiệm: mô tả môi trường, dữ liệu, cấu hình RAG, thiết kế fine-tuning, triển khai multi-agent và hệ thống quan sát.

Chương 5 – Kết quả và đánh giá: trình bày kết quả retrieval, kết quả sinh văn bản, so sánh các hướng tiếp cận, hiệu năng hệ thống, case study và hiệu quả kinh tế – xã hội.

Chương 6 – Kết luận và hướng phát triển: tóm tắt kết quả, nêu hạn chế và đề xuất hướng phát triển.

Phần cuối báo cáo gồm tài liệu tham khảo và các phụ lục (prompt mẫu, bảng cấu hình tham số và danh mục mô hình).

Chương 2

Cơ sở lý thuyết và tổng quan

2.1 Chatbot và trợ lý ảo

2.1.1 Khái niệm và phân loại

Chatbot là một chương trình máy tính được thiết kế để tương tác với con người thông qua hội thoại ngôn ngữ tự nhiên. Có thể phân loại chatbot theo nhiều tiêu chí. Theo mức độ phức tạp của cơ chế xử lý, chatbot được chia thành ba thế hệ:

- **Chatbot dựa trên luật (rule-based):** hoạt động theo các kịch bản và mẫu câu lệnh được định nghĩa trước. Ưu điểm là dễ kiểm soát và tiên đoán, nhưng cứng nhắc, khó mở rộng và dễ thất bại trước các cách diễn đạt ngoài kịch bản.
- **Chatbot dựa trên học máy (retrieval-based / intent-based):** sử dụng các mô hình phân loại ý định và trích xuất thực thể để ánh xạ câu hỏi người dùng vào một tập câu trả lời định sẵn. Linh hoạt hơn rule-based nhưng vẫn phụ thuộc vào tập ý định hữu hạn.
- **Chatbot sinh văn bản (generative) dựa trên mô hình ngôn ngữ lớn:** sinh câu trả lời trực tiếp từ mô hình ngôn ngữ, có khả năng diễn đạt tự nhiên và xử lý câu hỏi mở. Đây là hướng tiếp cận mà đề tài lựa chọn.

2.1.2 Trợ lý ảo và kiến trúc hội thoại

Khái niệm “trợ lý ảo” (virtual assistant) thường được hiểu rộng hơn chatbot: không chỉ trả lời câu hỏi mà còn có khả năng thực hiện tác vụ, gọi công cụ, truy cập hệ thống nghiệp vụ và duy trì ngữ cảnh qua nhiều lượt hội thoại. Một trợ lý ảo hiện đại thường gồm các thành phần: (i) bộ hiểu ngôn ngữ và quản lý hội thoại; (ii) bộ truy xuất tri thức; (iii) bộ gọi công cụ (tool calling); (iv) bộ sinh câu trả lời; và (v) lớp quan sát, ghi log để giám sát chất lượng.

Trong các hệ thống lớn, kiến trúc *multi-agent* (multi-agent) ngày càng phổ biến: một agent điều phối (orchestrator/host) tiếp nhận yêu cầu người dùng và định tuyến đến các agent chuyên biệt (specialist agents), mỗi agent phụ trách một miền nghiệp vụ hẹp. Cách tổ chức này giúp hệ thống dễ mở rộng, dễ kiểm thử và giảm tải nhận thức cho từng thành phần. Để các agent có thể giao tiếp và chia sẻ công cụ một cách chuẩn hóa, các giao thức như Model Context Protocol (MCP) và Agent-to-Agent (A2A) được sử dụng (trình bày ở mục 2.7).

2.2 Mô hình ngôn ngữ lớn

2.2.1 Kiến trúc Transformer

Hầu hết các mô hình ngôn ngữ lớn hiện đại đều dựa trên kiến trúc Transformer [1]. Khác với mạng hồi tiếp (RNN) xử lý tuần tự từng token, Transformer xử lý toàn bộ chuỗi song song nhờ cơ chế *tự chú ý* (self-attention), cho phép mô hình nắm bắt quan hệ giữa các token ở xa nhau một cách hiệu quả và song song hóa tốt trên GPU.

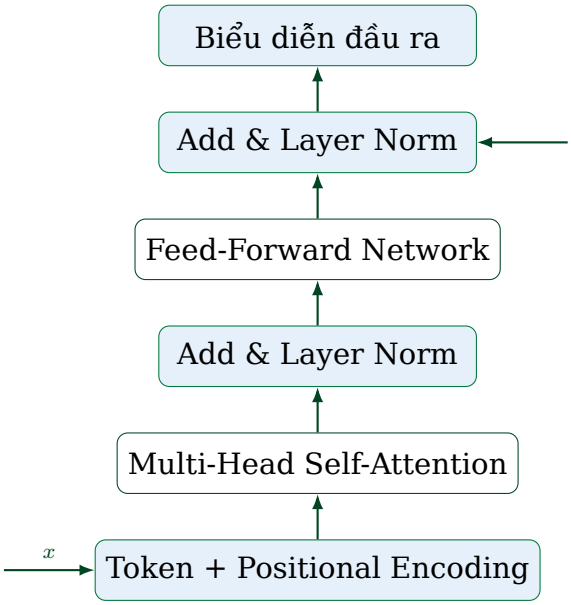
Cơ chế tự chú ý được mô tả bằng công thức *scaled dot-product attention*:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{2.1}$$

trong đó Q, K, V lần lượt là các ma trận truy vấn (query), khóa (key) và giá trị (value) được chiếu tuyến tính từ biểu diễn đầu vào, và d_k là số chiều của vector khóa. Cơ chế *đa đầu* (multi-head attention) chạy song song nhiều phép chú ý trên các không gian con khác nhau rồi ghép kết quả, giúp mô hình học nhiều loại quan hệ đồng thời. Mỗi khối Transformer còn gồm mạng truyền thẳng theo vị trí (position-wise feed-forward network), chuẩn hóa lớp (layer normalization) và kết nối tắt (residual connection). Thông tin vị trí của token được bổ sung qua positional encoding.

2.2.2 Tokenization và biểu diễn từ

Trước khi đưa vào mô hình, văn bản được chia thành các *token* bằng bộ tokenizer (thường theo thuật toán BPE [2] hoặc SentencePiece). Mỗi token được ánh xạ thành một vector embedding; các vector này được cập nhật qua nhiều lớp Transformer để tạo ra biểu diễn ngữ cảnh hóa. Chất lượng



Hình 2.1: Sơ đồ khối một khối Transformer (một lớp) với self-attention và feed-forward.

tokenizer ảnh hưởng trực tiếp đến khả năng xử lý tiếng Việt, đặc biệt với các từ ghép và dấu thanh.

2.2.3 Quá trình huấn luyện

Một mô hình ngôn ngữ lớn thường trải qua các giai đoạn:

1. **Tiền huấn luyện (pre-training):** huấn luyện mô hình dự đoán token tiếp theo trên khối lượng văn bản khổng lồ [3]. Giai đoạn này giúp mô hình học ngữ pháp, kiến thức phổ quát và khả năng lập luận ngôn ngữ.
2. **Fine-tuning có giám sát (instruction tuning/SFT):** tiếp tục huấn luyện trên các cặp hướng dẫn – đáp án để mô hình biết cách "trả lời" thay vì chỉ "hoàn thành văn bản" [4].
3. **Căn chỉnh theo phản hồi con người (RLHF) hoặc tối ưu theo sở thích (DPO):** điều chỉnh mô hình theo sở thích của con người về tính hữu ích, trung thực và an toàn [5, 6].

2.2.4 Sinh văn bản và prompt

Khi inference, mô hình sinh token lần lượt theo một chiến lược giải mã. Nhiệt độ (temperature) điều chỉnh độ ngẫu nhiên; top-k và top-p (nucleus sampling) giới hạn tập token được chọn, giúp cân bằng giữa tính đa dạng

và tính ổn định. Chất lượng đầu ra phụ thuộc rất lớn vào *prompt*: kỹ thuật zero-shot, few-shot (đưa ví dụ) và Chain-of-Thought (CoT) (Chain-of-Thought – CoT) [7] giúp định hướng mô hình giải quyết tác vụ.

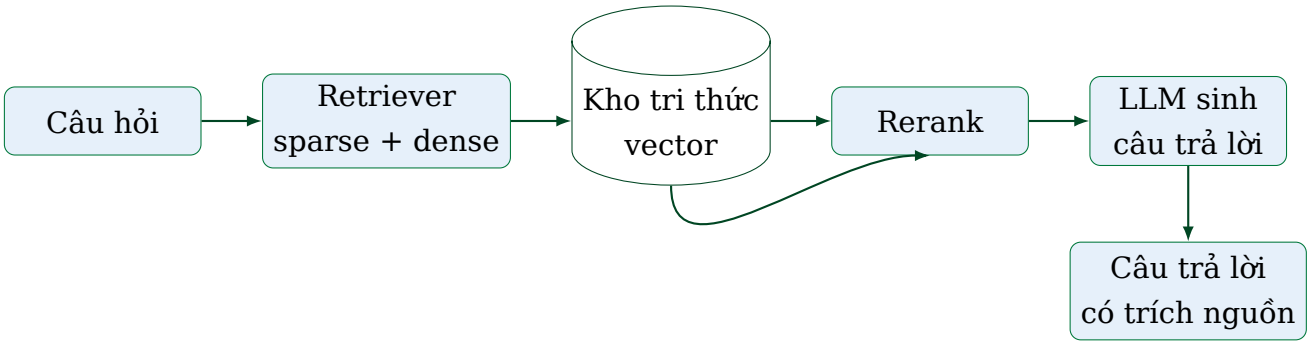
2.3 RAG (Retrieval-Augmented Generation)

2.3.1 Khái niệm và động lực

RAG [8] là kỹ thuật kết hợp một retriever với một mô hình sinh (generator). Thay vì buộc mô hình phải "nhớ" toàn bộ tri thức trong tham số, RAG lấy các document chunk liên quan từ kho tri thức ngoài tại thời điểm inference, rồi cung cấp chúng làm ngữ cảnh để mô hình sinh câu trả lời. Cách tiếp cận này mang lại ba lợi ích chính: (i) cập nhật tri thức mà không cần huấn luyện lại; (ii) giảm hiện tượng hallucination nhờ ràng buộc vào bằng chứng; và (iii) cho phép trích dẫn nguồn, tăng tính kiểm chứng.

2.3.2 Kiến trúc tổng quát

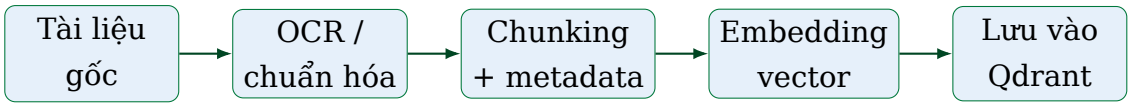
Một hệ RAG điển hình gồm hai pha: *indexing* và *inference*. Ở pha indexing, tài liệu được chuẩn hóa, chia thành chunk, embedding thành vector và lưu vào vector database. Ở pha inference, câu hỏi của người dùng được embedding, retriever tìm các chunk liên quan, chúng được rerank và ghép thành ngữ cảnh, cuối cùng mô hình sinh câu trả lời.



Hình 2.2: Kiến trúc tổng quát của một hệ thống RAG.

2.3.3 Chunking và tiền xử lý tài liệu

Chất lượng của RAG phụ thuộc trước hết vào cách tài liệu được chia thành các *chunk*. Chunk quá lớn làm loãng ngữ cảnh và tăng chi phí; chunk quá nhỏ làm mất mạch thông tin. Với tài liệu nghiệp vụ chứa nhiều bảng biểu và tiêu đề, chiến lược chunking thường dùng là *nhận biết cấu trúc* (header-aware): cắt theo tiêu đề mục, giữ nguyên bảng, đặt độ dài tối đa và độ chồng lấn (overlap) hợp lý để bảo toàn ngữ cảnh giữa các chunk liền kề. Với tài liệu scan hoặc ảnh, cần bước nhận dạng ký tự quang học (OCR) trước khi chunking.



Hình 2.3: Pipeline indexing tài liệu trong hệ thống RAG.

2.3.4 Embedding vector và vector database

Mô hình embedding ánh xạ text chunk thành một vector số thực trong không gian nhiều chiều, sao cho các chunk có ngữ nghĩa gần nhau nằm gần nhau. Độ tương đồng thường được đo bằng cosine:

$$\mathrm{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|}. \tag{2.2}$$

Vector database (vector database) lưu trữ và tìm kiếm láng giềng gần nhất trên hàng triệu vector một cách hiệu quả nhờ các thuật toán chỉ mục xấp xỉ [9]. Trong đề tài này, Qdrant [10] được sử dụng với khả năng lưu song song dense vector (dense) và sparse vector (sparse) trong cùng một collection.

2.3.5 Sparse retrieval, dense retrieval và hybrid retrieval

Sparse retrieval (sparse retrieval) biểu diễn văn bản bằng sparse vector dựa trên tần suất từ, tiêu biểu là BM25 [11]. Điểm BM25 giữa câu truy vấn q và tài liệu d được tính:

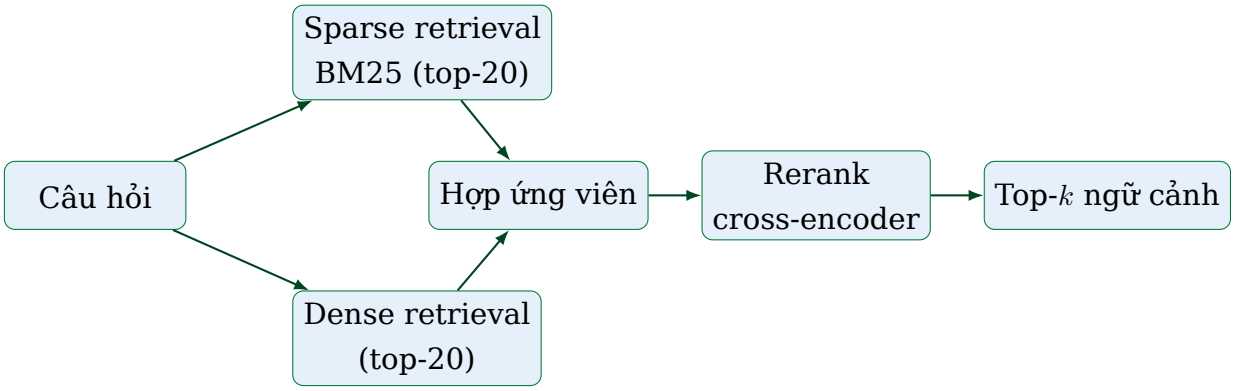
$$\mathrm{BM25}(q, d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t, d)\,(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}, \tag{2.3}$$

trong đó $f(t, d)$ là tần suất từ t trong d, $|d|$ là độ dài tài liệu, avgdl là độ dài trung bình, và k_1, b là các tham số điều chỉnh. BM25 mạnh ở việc khớp từ khóa chính xác, đặc biệt với số hiệu văn bản, mã dịch vụ hoặc thuật ngữ hiếm.

Dense retrieval (dense retrieval) [12] sử dụng embedding model để biểu diễn câu hỏi và tài liệu trong cùng không gian ngữ nghĩa, nhờ đó bắt được các trường hợp diễn đạt đồng nghĩa mà BM25 bỏ sót. Hai phương pháp có ưu nhược điểm bổ trợ cho nhau. *Hybrid retrieval* (hybrid retrieval) kết hợp cả hai bằng cách lấy hợp hoặc kết hợp điểm số, thường cho kết quả tốt hơn từng phương pháp riêng lẻ.

2.3.6 Rerank

Các retriever giai đoạn đầu (first-stage retrieval) thường nhanh nhưng kém chính xác do phải quét trên toàn kho. Bộ *reranker* sắp xếp lại thứ tự ứng viên bằng một mô hình mạnh hơn – thường là mô hình cross-encoder đọc đồng thời câu hỏi và chunk để cho điểm tương quan. Quy trình điển hình: lấy một tập ứng viên lớn ở giai đoạn một (ví dụ top-20 mỗi nhánh), hợp chúng lại, rồi rerank toàn bộ và giữ top-k nhỏ (ví dụ top-5) làm ngữ cảnh cho mô hình sinh.



Hình 2.4: Pipeline hybrid retrieval kết hợp BM25, dense retrieval và rerank.

2.3.7 Các biến thể của RAG

Từ kiến trúc cơ bản, RAG có nhiều biến thể [13]. *Naive RAG* là dạng đơn giản nhất với truy xuất một lần. *Advanced RAG* bổ sung tiền xử lý truy vấn, hậu xử lý kết quả và rerank. *Modular RAG* tổ chức thành các mô-đun linh hoạt có thể thay thế. Ngoài ra, *GraphRAG* khai thác đồ thị tri thức để truy

xuất theo quan hệ giữa các thực thể, và *agentic RAG* cho phép agent quyết định truy xuất nhiều vòng, cải viết truy vấn hoặc gọi công cụ khác khi cần. Đề tài này áp dụng advanced/modular RAG với hybrid retrieval và rerank, đồng thời định hướng mở rộng lên agentic và GraphRAG trong tương lai.

2.4 Fine-tuning mô hình ngôn ngữ lớn (Fine-tuning)

2.4.1 Khái niệm và các mức độ

Fine-tuning (fine-tuning) là quá trình tiếp tục huấn luyện một mô hình đã được tiền huấn luyện trên một dataset chuyên biệt, nhằm điều chỉnh mô hình phù hợp với miền hoặc tác vụ cụ thể. Có thể chia thành hai nhóm:

- **Full fine-tuning:** cập nhật toàn bộ tham số mô hình. Hiệu quả cao nhưng đòi hỏi tài nguyên tính toán và bộ nhớ rất lớn, đồng thời cần lưu toàn bộ bản sao mô hình cho mỗi tác vụ.
- **Fine-tuning hiệu quả tham số (PEFT):** chỉ cập nhật một phần nhỏ tham số, giữ nguyên phần lớn mô hình gốc. Tiêu biểu là LoRA và QLoRA.

2.4.2 Fine-tuning có giám sát và căn chỉnh

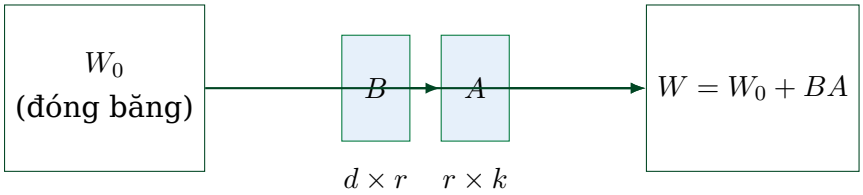
Fine-tuning có giám sát (Supervised Fine-Tuning – SFT) huấn luyện mô hình trên các cặp hướng dẫn – đáp án chất lượng cao. Để mô hình trả lời đúng "giọng" nghiệp vụ, dữ liệu SFT cần được thiết kế cẩn thận cả về nội dung lẫn định dạng. Sau SFT, có thể áp dụng RLHF hoặc DPO [6] để căn chỉnh theo sở thích con người, giúp câu trả lời hữu ích và an toàn hơn.

2.4.3 LoRA

LoRA (Low-Rank Adaptation) [14] giả định rằng thay đổi tham số khi thích ứng một tác vụ có "hạng thấp" (low intrinsic rank). Thay vì cập nhật toàn bộ ma trận trọng số $W_0 \in \mathbb{R}^{d\times k}$, LoRA giữ nguyên W_0 và thêm một phần hiệu chỉnh phân rã thành tích hai ma trận hạng thấp:

$$W = W_0 + \Delta W = W_0 + BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d,k). \tag{2.4}$$

Ma trận A được khởi tạo ngẫu nhiên và B được khởi tạo bằng 0 để lúc bắt đầu $\Delta W = 0$, bảo toàn hành vi mô hình gốc. Số tham số huấn luyện giảm từ $d \times k$ xuống $r(d+k)$, thường chỉ còn dưới 1% tổng tham số. Hệ số α và hạng r quyết định độ lớn của cập nhật; giá trị hiệu chỉnh thường được nhân với α/r.



Hình 2.5: Minh họa LoRA: giữ nguyên W_0 và thêm cập nhật hạng thấp BA.

2.4.4 QLoRA

QLoRA [15] mở rộng LoRA bằng cách quantization mô hình gốc xuống 4 bit (định dạng NF4), áp dụng quantization kép (double quantization) và tối ưu hóa phân trang (paged optimizers). Nhờ đó có thể fine-tuning các mô hình lớn trên phần cứng khiêm tốn hơn đáng kể so với full fine-tuning, với mức suy giảm chất lượng nhỏ.

2.4.5 Khi nào nên dùng fine-tuning

Fine-tuning phù hợp khi cần điều chỉnh *hành vi* của mô hình (phong cách, định dạng, quy trình nghiệp vụ ổn định) hoặc khi tri thức miền mang tính ổn định và khó mô tả bằng ngữ cảnh retrieval. Ngược lại, khi tri thức thay đổi thường xuyên hoặc cần trích dẫn nguồn, RAG là lựa chọn phù hợp hơn.

2.5 So sánh RAG và fine-tuning, kiến trúc hybrid

RAG và fine-tuning giải quyết hai lớp vấn đề khác nhau và có thể bổ trợ cho nhau. Bảng 2.1 tóm tắt sự so sánh.

Từ so sánh trên, đề tài đề xuất *kiến trúc hybrid* kết hợp cả hai: sử dụng RAG làm kênh cung cấp tri thức động và bằng chứng, đồng thời dùng mô hình đã fine-tuning để sinh câu trả lời đúng phong cách, đúng định dạng

Bảng 2.1: So sánh RAG và fine-tuning theo các tiêu chí triển khai.

Tiêu chí	**RAG**	**Fine-tuning**
Nguồn tri thức	Kho tài liệu ngoài, truy xuất động	Tham số mô hình, cố định sau huấn luyện
Cập nhật tri thức	Chỉ cần cập nhật chỉ mục	Cần huấn luyện lại
Khả năng trích nguồn	Có (dựa trên chunk retrieval)	Hạn chế
Chi phí huấn luyện	Thấp (chủ yếu indexing)	Cao hơn (SFT/LoRA)
Chi phí inference	Cao hơn (thêm retrieval)	Thấp hơn (chỉ sinh)
Điều chỉnh phong cách/định dạng	Yếu	Mạnh
Giảm hallucination	Tốt nếu truy xuất đúng	Trung bình
Phù hợp nhất với	Tri thức động, tra cứu	Hành vi miền ổn định

nghiệp vụ và xử lý các tình huống ổn định. Một router quyết định khi nào chỉ cần RAG, khi nào dùng mô hình fine-tuning, và khi nào kết hợp. Chi tiết kiến trúc được trình bày ở chương 3.

2.6 Các thước đo đánh giá

Đánh giá một hệ RAG cần tách thành hai khâu: chất lượng retrieval và chất lượng sinh.

2.6.1 Thước đo retrieval

- **Recall@k:** tỉ lệ các tài liệu/chunk liên quan được tìm thấy trong top-k. Đây là thước đo quan trọng nhất vì nếu tài liệu đúng không nằm trong ngữ cảnh thì mô hình không thể trả lời đúng.
- **Precision@k:** tỉ lệ chunk liên quan trong top-k.
- **MRR (Mean Reciprocal Rank):** nghịch đảo thứ hạng của chunk liên quan đầu tiên, trung bình trên các truy vấn. MRR cao nghĩa là chunk đúng xuất hiện sớm.
- **nDCG:** thước đo có trọng số theo vị trí, phù hợp khi đánh giá chất lượng xếp hạng với mức độ liên quan khác nhau.

2.6.2 Thước đo sinh văn bản

Bộ thước đo RAGAS [16] đánh giá hệ RAG không cần đáp án tham chiếu đầy đủ:

- **Faithfulness (độ trung thực):** mức độ câu trả lời được hỗ trợ bởi ngữ cảnh retrieval, phản ánh mức độ hallucination.
- **Answer Relevancy (độ liên quan câu trả lời):** mức độ câu trả lời đúng trọng tâm câu hỏi.
- **Context Precision:** tỉ lệ context chunk thực sự hữu ích.
- **Context Recall:** mức độ ngữ cảnh bao phủ đáp án tham chiếu.

Ngoài ra, *LLM-as-judge* sử dụng một mô hình ngôn ngữ mạnh để cho điểm câu trả lời theo thang điểm (ví dụ 0–10) dựa trên đáp án chuẩn, là phương pháp linh hoạt và tiết kiệm cho miền hẹp. Các thước đo truyền thống như BLEU và ROUGE có thể dùng bổ sung nhưng không phản ánh tốt tính đúng đắn ngữ nghĩa trong hỏi đáp mở.

2.7 Nền tảng triển khai

2.7.1 Phục vụ inference với vLLM

vLLM [17, 18] là thư viện LLM serving LLM hiệu năng cao, nổi bật với cơ chế PagedAttention quản lý bộ nhớ KV-cache theo trang, cho phép tăng throughput đáng kể nhờ xử lý theo lô liên tục (continuous batching). Kết hợp với quantization FP8, vLLM cho phép phục vụ các mô hình hàng chục tỷ tham số trên GPU với chi phí hợp lý.

2.7.2 Quantization FP8

FP8 là định dạng số dấu phẩy động 8 bit, giảm một nửa dung lượng bộ nhớ so với FP16 và tăng tốc tính toán trên phần cứng hỗ trợ, với mức suy giảm chất lượng thường nhỏ. Đây là lựa chọn phổ biến để phục vụ mô hình lớn trên hạ tầng giới hạn.

2.7.3 Giao thức MCP và A2A

Model Context Protocol (MCP) [19] chuẩn hóa cách mô hình truy cập công cụ và nguồn dữ liệu, giúp tách biệt logic công cụ khỏi agent. **Agent-**

to-Agent (A2A) [20] chuẩn hóa giao tiếp giữa các agent, cho phép một agent điều phối gọi agent chuyên biệt và nhận kết quả. Hai giao thức này tạo nên nền tảng cho kiến trúc multi-agent của đề tài.

2.7.4 Quan sát hệ thống và quản lý prompt

OpenTelemetry (OTel) [21] cung cấp chuẩn thu thập vết (trace), chỉ số (metric) và log; dữ liệu được đẩy tới các nền tảng như Langfuse [22] hoặc Grafana Tempo để giám sát inference flow, latency và lượng token. MLflow Prompt Registry [23] cho phép quản lý phiên bản prompt tập trung, giúp tách cấu hình khỏi mã nguồn và kiểm soát thay đổi prompt theo môi trường (ví dụ nhãn `@production`).

Kết chương

Chương này đã hệ thống hóa nền tảng lý thuyết về chatbot, LLM, RAG, fine-tuning, kiến trúc lai, thước đo đánh giá và nền tảng triển khai. Đây là cơ sở để chương tiếp theo phân tích bài toán cụ thể của VNPost và thiết kế kiến trúc hệ thống đề xuất.

Chương 3

Kiến trúc hệ thống đề xuất

Trên nền tảng lý thuyết đã trình bày ở chương 2, chương này phân tích bài toán cụ thể của VNPost và đề xuất kiến trúc trợ lý ảo. Mục 3.1 làm rõ đặc thù bài toán; mục 3.2 xác định yêu cầu; mục 3.3 trình bày kiến trúc multi-agent; mục 3.4 và mục 3.5 mô tả pipeline RAG và pipeline fine-tuning; mục 3.6 đề xuất kiến trúc hybrid; và mục 3.7 trình bày hạ tầng và bảo mật.

3.1 Đặc thù bài toán tại VNPost

3.1.1 Đặc điểm dữ liệu nội bộ

Dữ liệu nội bộ của VNPost có một số đặc điểm ảnh hưởng trực tiếp đến thiết kế hệ thống:

- **Đa dạng định dạng:** tài liệu tồn tại dưới dạng văn bản số (docx, pdf, txt) và cả bản scan. Điều này đòi hỏi bước nhận dạng ký tự quang học và chuẩn hóa về một định dạng thống nhất trước khi indexing.
- **Cấu trúc phức tạp:** nhiều văn bản chứa bảng biểu (biểu phí, thời gian toàn trình, bảng phân loại hàng hóa), tiêu đề nhiều cấp và số hiệu văn bản. Việc chunking phải bảo toàn bảng và ngữ cảnh tiêu đề.
- **Ngôn ngữ chuyên ngành:** nhiều thuật ngữ, từ viết tắt và số hiệu đặc thù (ví dụ số quyết định, mã dịch vụ), đòi hỏi cấu hình sparse retrieval phù hợp với tiếng Việt.
- **Tính động:** biểu phí và quy định thay đổi theo thời gian, nên tri thức cần được cập nhật mà không phải huấn luyện lại mô hình.
- **Tính bảo mật:** tài liệu nội bộ và dữ liệu khách hàng không được phép rời khỏi hạ tầng của Tổng công ty.

3.1.2 Các nhóm nghiệp vụ

Hệ thống phục vụ ba nhóm nghiệp vụ chính: (i) hỏi đáp trên tài liệu văn thư và sản phẩm – dịch vụ; (ii) tra cứu tình trạng bưu gửi; và (iii) tra cứu thông tin bưu cục. Ngoài ra còn các nghiệp vụ mở rộng như tra cứu chất lượng phát và tra cứu tài chính – kế toán.

3.2 Yêu cầu chức năng và phi chức năng

3.2.1 Yêu cầu chức năng

- Trả lời câu hỏi tự nhiên dựa trên tài liệu nội bộ, có trích dẫn nguồn.
- Ngữ cảnh hóa câu trả lời theo lịch sử hội thoại nhiều lượt.
- Tra cứu tình trạng bưu gửi và thông tin bưu cục qua kết nối hệ thống nghiệp vụ.
- Định tuyến yêu cầu đến agent chuyên biệt phù hợp và hợp nhất kết quả.
- Hỗ trợ tạo phiếu yêu cầu (ticket) khi cần chuyển sang hỗ trợ thủ công.
- Gợi ý câu hỏi mở đầu phù hợp với nhu cầu người dùng.

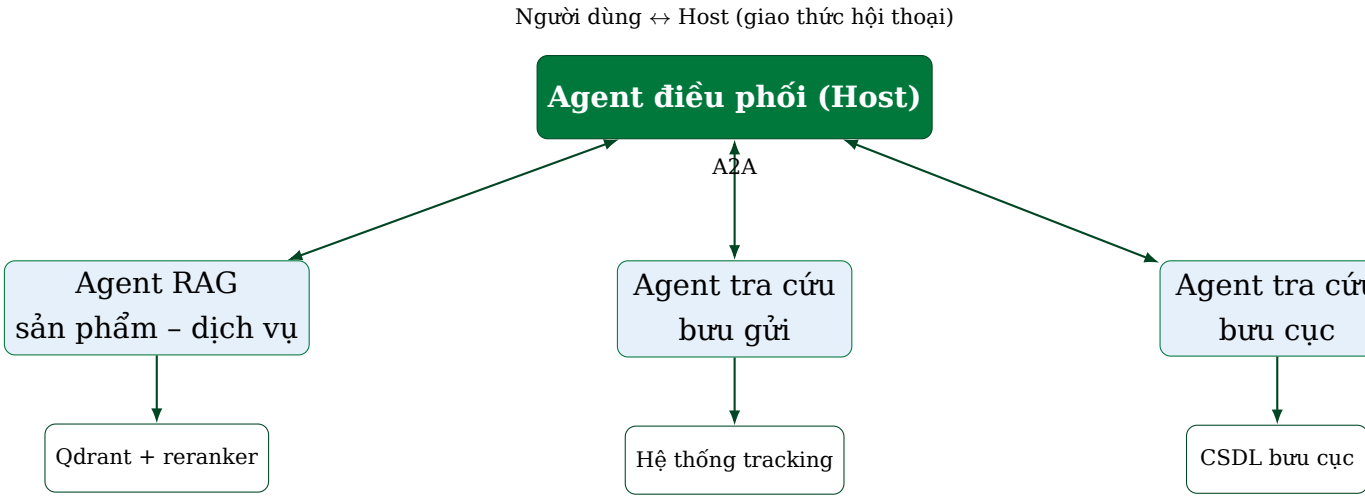
3.2.2 Yêu cầu phi chức năng

- **Chính xác và có căn cứ:** câu trả lời phải bám sát tài liệu, hạn chế tối đa hallucination.
- **Bảo mật:** toàn bộ mô hình và dữ liệu vận hành cục bộ, không gửi ra ngoài.
- **Latency chấp nhận được:** phản hồi trong vài giây cho câu hỏi thông thường.
- **Khả năng mở rộng:** dễ thêm agent, kho tài liệu và mô hình mới.
- **Quan sát được:** có vết, log và chỉ số để giám sát và cải tiến.
- **Kiểm soát truy cập:** phân quyền theo vai trò đối với dữ liệu nhạy cảm.

3.3 Kiến trúc tổng thể multi-agent

Đề tài đề xuất kiến trúc *multi-agent* gồm một agent điều phối trung tâm (host orchestrator) và các agent chuyên biệt. Agent điều phối chịu trách

nhiệm hiểu yêu cầu, duy trì ngữ cảnh hội thoại, quyết định định tuyến và hợp nhất câu trả lời. Các agent chuyên biệt đảm nhận từng miền nghiệp vụ hẹp và được gọi qua giao thức A2A; mỗi agent truy cập công cụ của mình qua giao thức MCP.



Hình 3.1: Kiến trúc multi-agent: host điều phối các agent chuyên biệt qua A2A.

Ưu điểm của kiến trúc này là tách biệt trách nhiệm rõ ràng: mỗi agent có prompt, công cụ và kho dữ liệu riêng, dễ kiểm thử và nâng cấp độc lập. Agent điều phối chỉ cần biết "khi nào gọi ai", nhờ đó giảm tải nhận thức và tăng độ ổn định.

3.4 Pipeline RAG chi tiết

3.4.1 Pha indexing

Pha indexing biến tài liệu thô thành các chunk vector trong Qdrant qua các bước:

1. **Thu thập và chuẩn hóa:** tài liệu được thu thập, với các bản scan được xử lý OCR bằng mô hình nhận dạng (ví dụ `olmocr-2-7b` hoặc `lightonocr2-1b`).
2. **Chuyển sang Markdown:** tài liệu được chuyển thành văn bản có cấu trúc để bảo toàn tiêu đề và bảng.
3. **Trích xuất metadata:** mỗi tài liệu được gắn metadata như tiêu đề, số hiệu văn bản, ngày ban hành và tóm tắt do mô hình ngôn ngữ sinh ra.

4. **Chunking:** văn bản được chia thành các chunk nhận biết cấu trúc, với độ dài tối đa và độ chồng lấn hợp lý, bảo toàn bảng biểu.
5. **Embedding và lưu trữ:** mỗi chunk được biểu diễn thành embedding vector bằng embedding model và lưu vào Qdrant, cùng sparse vector cho BM25.

3.4.2 Pha retrieval và sinh

Khi có câu hỏi, hệ thống thực hiện:

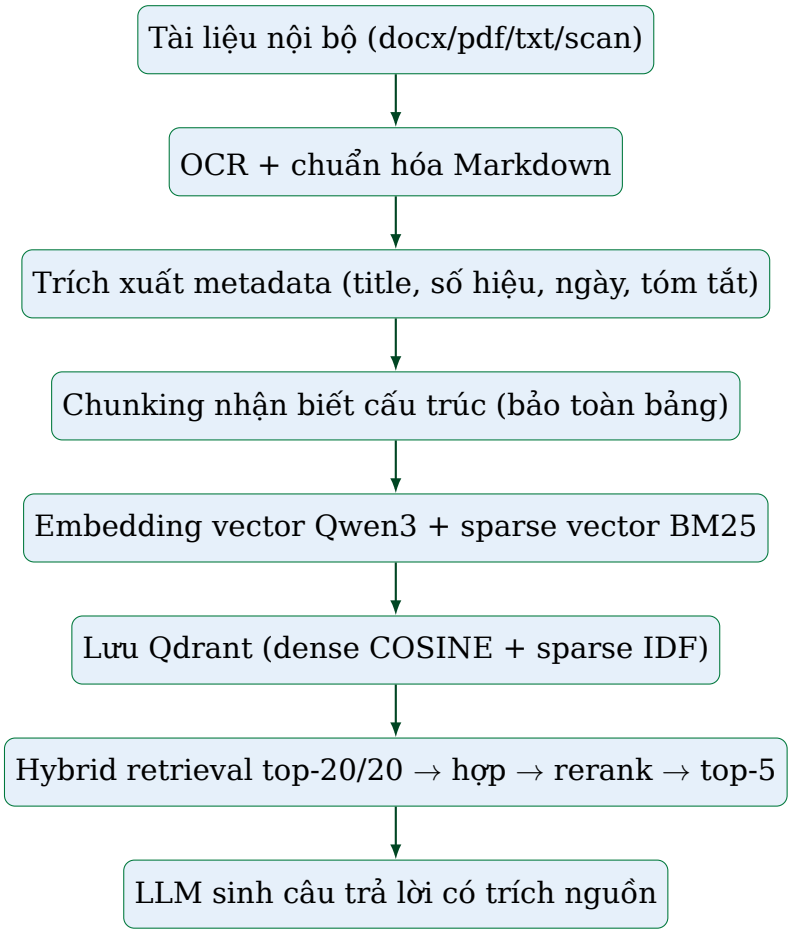
1. **Embedding câu hỏi** bằng cùng embedding model đã dùng khi indexing.
2. **Hybrid retrieval:** chạy song song sparse retrieval BM25 và dense retrieval, mỗi nhánh lấy top-k_1 ứng viên (ví dụ 20).
3. **Hợp ứng viên:** gộp kết quả hai nhánh và loại trùng.
4. **Rerank:** dùng cross-encoder rerank toàn bộ ứng viên và giữ top-k_2 (ví dụ 5) làm ngữ cảnh.
5. **Sinh câu trả lời:** mô hình ngôn ngữ nhận câu hỏi, ngữ cảnh retrieval và lịch sử hội thoại để sinh câu trả lời có trích nguồn.

3.4.3 Tham số hóa và cấu hình

Các tham số chính gồm số ứng viên mỗi nhánh retrieval (k_1), số chunk sau rerank (k_2) và ngưỡng điểm tối thiểu để loại bỏ ngữ cảnh không liên quan. Cấu hình thực tế được trình bày ở chương 4. Cơ chế ngưỡng điểm giúp hệ thống từ chối trả lời khi không tìm thấy bằng chứng phù hợp, thay vì "bịa" câu trả lời.

3.5 Pipeline fine-tuning đề xuất

Vì codebase hiện tại của VNPost chưa có pipeline fine-tuning, phần này trình bày *thiết kế đề xuất*. Trong một hệ RAG, có *ba loại mô hình* có thể được fine-tune với mục tiêu khác nhau: (i) **LLM** sinh câu trả lời – điều chỉnh hành vi, phong cách và định dạng nghiệp vụ; (ii) **embedding model** – cải thiện chất lượng biểu diễn để dense retrieval chính xác hơn; và (iii) **reranker** – cải thiện xếp hạng để giữ đúng chunk trong top-k. Hai loại sau tác động trực tiếp vào khâu retrieval/rerank, vốn là điểm nghẽn được chỉ ra ở chương 5.



Hình 3.2: Pipeline RAG đầy đủ từ indexing đến sinh câu trả lời.

Hiện tại, hệ thống đang dùng các mô hình embedding và reranker *nguyên bản (pre-trained, off-the-shelf)*; việc fine-tune chúng là nội dung đề xuất của báo cáo.

3.5.1 Thu thập và chuẩn bị dữ liệu

Dữ liệu huấn luyện được tổng hợp từ ba nguồn: (i) log hội thoại thực tế giữa người dùng và hệ thống, sau khi lọc và ẩn thông tin định danh cá nhân (PII); (ii) các cặp hỏi – đáp chuẩn do chuyên gia nghiệp vụ xây dựng; và (iii) dữ liệu tổng hợp do mô hình mạnh sinh ra có kiểm duyệt. Tùy loại mô hình, dữ liệu được định dạng khác nhau:

- **Với LLM:** định dạng theo mẫu hội thoại (instruction – response).
- **Với embedding và reranker:** định dạng theo cặp truy vấn – chunk, trong đó chunk chứa đáp án là mẫu dương (positive), còn các chunk được truy hồi nhưng không chứa đáp án là mẫu âm (negative). Các mẫu âm *khó* (hard negatives) được khai thác trực tiếp từ kết quả truy hồi sai của BM25/dense, giúp mô hình học phân biệt tốt hơn.

Dữ liệu được chia thành tập huấn luyện, tập kiểm định và tập kiểm thử *theo câu hỏi* để tránh rò rỉ dữ liệu giữa các tập.

3.5.2 Fine-tuning LLM (LoRA/QLoRA)

Đề xuất sử dụng LoRA hoặc QLoRA với hạng $r = 16$, hệ số $\alpha = 32$, dropout nhỏ, và áp dụng lên các ma trận chiếu trong khối attention và feed-forward (query, key, value, output, gate, up, down). Mô hình gốc được giữ ở định dạng quantization 4 bit khi dùng QLoRA để tiết kiệm bộ nhớ. Sau huấn luyện, adapter có thể được gộp (merge) vào mô hình gốc hoặc phục vụ như một adapter riêng trong vLLM. Mục tiêu là điều chỉnh *hành vi* chứ không nạp tri thức (tri thức do RAG đảm nhiệm).

3.5.3 Fine-tuning embedding model

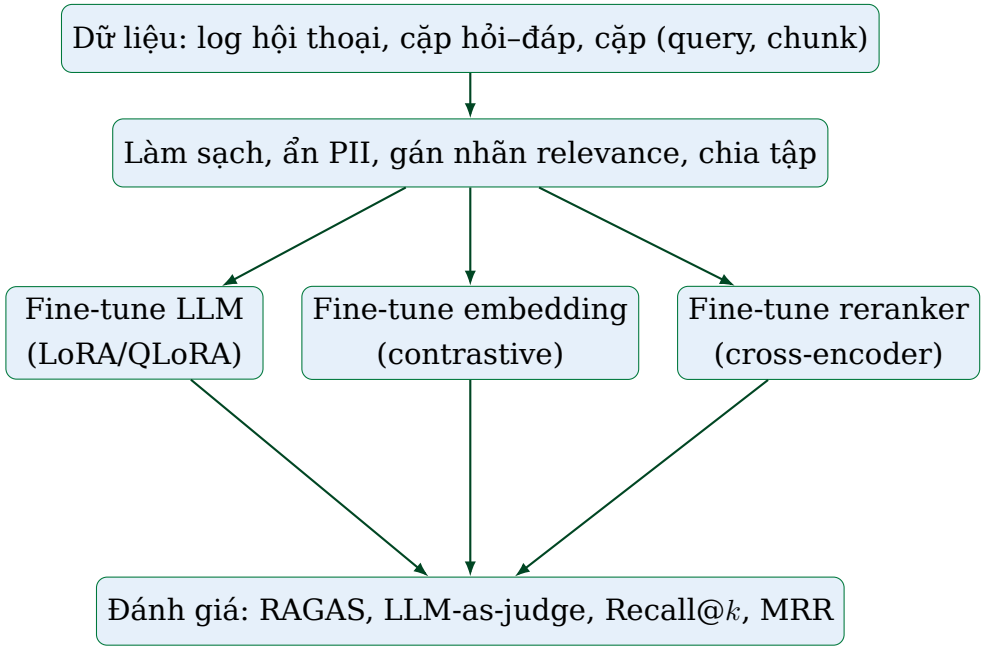
Mục tiêu là tăng Recall@k của dense retrieval bằng cách đưa các biểu diễn của câu hỏi và chunk chứa đáp án lại gần nhau hơn. Phương pháp đề xuất là *học tương phản* (contrastive learning) với hàm mất mát xếp hạng nhiều mẫu âm (Multiple Negatives Ranking Loss): mỗi cặp truy vấn – chunk dương được đối chiếu với các mẫu âm trong cùng batch (in-batch negatives) và các

hard negatives. Mô hình nền là Qwen3-Embedding; có thể fine-tune toàn bộ hoặc chỉ tinh chỉnh lớp chiếu. Chỉ dẫn (instruction) chỉ áp dụng cho phía truy vấn, giữ nhất quán với lúc suy luận.

Sau khi thay embedding model, cần *đánh giá lại toàn bộ chỉ mục* (re-embed corpus) và đo lại Recall@k, MRR để xác nhận cải thiện trước khi đưa vào sản xuất.

3.5.4 Fine-tuning reranker

Reranker là cross-encoder nhận đồng thời (truy vấn, chunk) và cho điểm mức độ liên quan. Fine-tune reranker nhằm cải thiện Recall@k sau rerank – hiện chỉ đạt 50% ở cấu hình sản xuất – nên đây là hướng xử lý trực tiếp điểm nghẽn. Dữ liệu là các cặp (truy vấn, chunk) được gán nhãn liên quan theo đáp án tham chiếu (nhãn nhị phân hoặc theo mức), với hard negatives lấy từ top-k sai. Huấn luyện bằng hàm mất mát phân loại nhị phân (cross-entropy) hoặc xếp hạng (margin/ranking loss). Mô hình nền là Qwen3-Reranker; có thể fine-tune toàn bộ hoặc dùng LoRA. Đánh giá bằng Recall@5, MRR và nDCG.



Hình 3.3: Pipeline fine-tuning đề xuất cho ba loại mô hình: LLM, embedding và reranker.

Ghi chú minh bạch: pipeline fine-tuning trong báo cáo là thiết kế đề xuất*; các kết quả liên quan ở chương 5 (nếu có) được ghi rõ là kết quả*

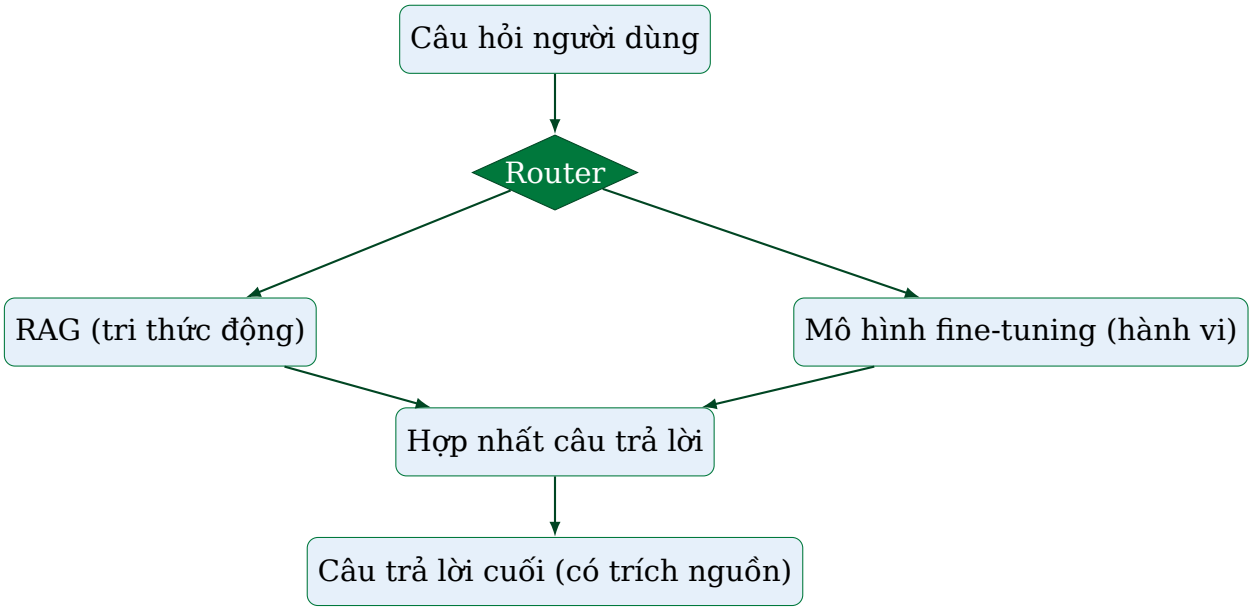
minh họa, không phải kết quả huấn luyện thật trên hạ tầng VNPost.

3.6 Kiến trúc hybrid RAG + Fine-tuning

Kiến trúc hybrid sử dụng một router để quyết định chiến lược trả lời:

- **Chỉ RAG:** với câu hỏi cần tri thức động hoặc cần trích dẫn nguồn (biểu phí, quy định, số hiệu văn bản). Đây là chế độ mặc định cho phần lớn truy vấn.
- **Chỉ mô hình fine-tuning:** với câu hỏi về phong cách, định dạng, hướng dẫn quy trình ổn định, hoặc khi cần câu trả lời ngắn gọn theo mẫu nghiệp vụ.
- **Kết hợp:** mô hình fine-tuning sinh khung câu trả lời theo định dạng nghiệp vụ, sau đó RAG bổ sung bằng chứng và số liệu cụ thể; mô hình hợp nhất tạo câu trả lời cuối.

Router có thể dựa trên phân loại ý định nhẹ, hoặc để chính agent điều phối quyết định dựa trên prompt. Khi RAG không tìm thấy bằng chứng đủ tin cậy, hệ thống chuyển sang thông báo thiếu thông tin hoặc tạo ticket, tránh sinh câu trả lời không căn cứ.



Hình 3.4: Kiến trúc hybrid: router chọn RAG, mô hình fine-tuning hoặc kết hợp.

3.7 Hạ tầng và bảo mật

3.7.1 Hạ tầng phục vụ

Toàn bộ hệ thống được triển khai trên Kubernetes để bảo đảm khả năng mở rộng và ổn định. Các mô hình được phục vụ cục bộ bằng vLLM với quantization FP8. Các mô hình chính gồm: mô hình sinh ngôn ngữ (ví dụ họ Qwen với các biến thể 27B–35B), embedding model văn bản (ví dụ Qwen3-Embedding-8B), reranker (Qwen3-Reranker-8B) và mô hình OCR. Qdrant đảm nhiệm lưu trữ vector.

3.7.2 Bảo mật dữ liệu

Yêu cầu bảo mật được thực thi ở nhiều lớp: (i) mô hình chạy cục bộ, dữ liệu không rời hạ tầng; (ii) kiểm soát truy cập theo vai trò (RBAC) đối với dữ liệu nhạy cảm; (iii) ẩn PII trong log trước khi dùng cho phân tích hoặc fine-tuning; và (iv) phân quyền ở tầng dữ liệu, bảo đảm người dùng chỉ truy cập được phạm vi cho phép.

Kết chương

Chương này đã phân tích bài toán VNPost, xác định yêu cầu và đề xuất kiến trúc multi-agent kết hợp RAG lai với fine-tuning theo mô hình lai. Chương tiếp theo mô tả việc triển khai thực nghiệm các thành phần này cùng các tham số cụ thể.

Chương 4

Triển khai thực nghiệm

Chương này trình bày việc triển khai hệ thống theo kiến trúc đã đề xuất ở chương 3. Mục 4.1 mô tả môi trường và công cụ; mục 4.2 mô tả dữ liệu; mục 4.3 trình bày cấu hình RAG; mục 4.4 trình bày thiết kế fine-tuning; mục 4.5 mô tả triển khai multi-agent; mục 4.6 trình bày hệ thống quan sát và quản lý prompt; và mục 4.7 tổng kết khó khăn cùng giải pháp.

4.1 Môi trường và công cụ

Hệ thống được triển khai theo mô hình microservice trên nền Kubernetes, đóng gói bằng Docker. Bảng 4.1 liệt kê các công cụ chính.

Bảng 4.1: Các công cụ và thành phần chính của hệ thống.

Thành phần	Vai trò
vLLM	Phục vụ inference LLM, embedding, reranker và OCR với FP8
Qdrant	Vector database, lưu dense vector và sparse
Kubernetes	Điều phối container, mở rộng và ổn định dịch vụ
Docker Compose	Triển khai cục bộ phục vụ phát triển và thử nghiệm
MLflow	Quản lý phiên bản prompt và gateway
Langfuse / Grafana Tempo	Thu thập và trực quan hóa inference trace
OpenTelemetry	Chuẩn thu thập trace, metric, log
Airflow	Lập lịch thu thập và cập nhật dữ liệu định kỳ
PostgreSQL	Lưu trữ dữ liệu nghiệp vụ (bưu cục, tracking)

4.2 Dữ liệu

4.2.1 Các kho tài liệu

Đề tài sử dụng ba kho tài liệu đại diện cho các miền nghiệp vụ khác nhau, tóm tắt ở bảng 4.2.

Bảng 4.2: Thống kê các kho tài liệu sử dụng trong thực nghiệm.

Kho tài liệu	Số tài liệu	Nội dung tiêu biểu
Văn thư nội bộ	17	Quyết định, công văn, quy trình nghiệp vụ
Sản phẩm – dịch vụ	119	Biểu phí, quy định hàng hóa, bồi thường, thời gian toàn trình
Chuyển phát nhanh EMS	≈ 30	Biểu phí EMS, quy định hàng hóa đặc biệt, bồi thường

Kho tài liệu sản phẩm – dịch vụ gồm 114 tệp văn bản, 3 tệp PDF và 2 tệp docx; sau khi chuẩn hóa, mỗi tài liệu được sinh thêm một tệp Markdown và gắn metadata (tiêu đề, số hiệu, ngày, tóm tắt). Kho văn thư gồm các quyết định và công văn về cước phí, thời gian toàn trình, hàng hóa cấm gửi và chính sách bồi thường.

4.2.2 Bộ đánh giá

Bộ đánh giá cho miền EMS gồm 99 cặp câu hỏi – đáp án tham chiếu, được trích từ một bảng gồm hơn 400 dòng dữ liệu thô và lọc lại còn 99 cặp sử dụng được. Bộ này được dùng cho cả đánh giá retrieval lẫn đánh giá sinh văn bản. Ngoài ra, một bộ mẫu nhỏ (6 câu hỏi) được dùng để đánh giá retrieval chi tiết theo từng nhánh (BM25, dense) và một bộ mẫu 3 câu cho đánh giá agent bằng LLM-as-judge.

4.2.3 Tiền xử lý

Quy trình tiền xử lý gồm: OCR cho bản scan; chuyển đổi sang Markdown; trích xuất metadata; structure-aware chunking; và sinh file băm (hash) để theo dõi thay đổi tài liệu. Đối với tiếng Việt, cấu hình sparse retrieval được

điều chỉnh: tắt stemming và loại bỏ danh sách từ dừng, vì stemming tiếng Việt kém hiệu quả với thuật ngữ chuyên ngành và số hiệu văn bản.

4.3 Cài đặt RAG

4.3.1 Chunking và embedding

Văn bản được chia thành các chunk với độ dài tối đa khoảng 1000 ký tự và độ chồng lấn bảo toàn ngữ cảnh. Bộ chunking ưu tiên ranh giới theo tiêu đề và giữ nguyên bảng. Mỗi chunk được embedding bằng mô hình Qwen3-Embedding với vector 4096 chiều; song song đó, sparse vector cho BM25 được tạo với chỉ số IDF.

4.3.2 Cấu hình retrieval

Bảng 4.3 trình bày các tham số retrieval chính. Ở nhánh sản phẩm – dịch vụ của hệ thống multi-agent, số ứng viên BM25 là 10 và số ứng viên sau rerank là 10; ở cấu hình mẫu (template) dùng cho đánh giá retrieval, các giá trị tương ứng là 20/20/5.

Bảng 4.3: Cấu hình tham số retrieval RAG.

Tham số	**Cấu hình mẫu**	**Nhánh SPDV**
Số ứng viên dense (k_1^{dense})	20	20
Số ứng viên BM25 (k_1^{bm25})	20	10
Số chunk sau rerank (k_2)	5	10
Ngưỡng điểm rerank	0,0	0,0
Số chiều embedding	4096	4096
Độ dài chunk tối đa	≈ 1000	≈ 1000

4.3.3 Embedding câu hỏi và prompt

Câu hỏi được embedding với chỉ dẫn (instruction) phù hợp cho retrieval. Prompt hệ thống của agent RAG được quản lý tập trung qua MLflow Prompt Registry (ví dụ tham chiếu `prompts:/tct-rag-agent@production`), cho phép cập nhật mà không cần triển khai lại mã. Prompt quy định rõ: chỉ trả lời dựa trên ngữ cảnh retrieval, trích nguồn, và từ chối khi thiếu thông tin.

4.4 Thiết kế fine-tuning

Như đã nêu, đây là phần *thiết kế đề xuất* cho cả ba loại mô hình. Bảng 4.4 trình bày cấu hình LoRA/QLoRA cho LLM; bảng 4.5 cho embedding model; và bảng 4.6 cho reranker. Hiện tại embedding model và reranker đang dùng nguyên bản; các cấu hình dưới đây là đề xuất để cải thiện khâu retrieval và rerank.

Bảng 4.4: Cấu hình LoRA/QLoRA đề xuất cho fine-tuning LLM.

Tham số	**Giá trị đề xuất**
Phương pháp	LoRA / QLoRA (4-bit NF4)
Hạng r	16
Hệ số α	32
Dropout	0,05
Module áp dụng	q, k, v, o, gate, up, down
Tốc độ học	1×10^{-4} đến 2×10^{-4}
Số epoch	2–3
Độ dài ngữ cảnh	4096 token
Chiến lược	SFT, có thể bổ sung DPO

Bảng 4.5: Cấu hình fine-tuning embedding model đề xuất.

Tham số	**Giá trị đề xuất**
Mô hình nền	Qwen3-Embedding (4096 chiều)
Phương pháp	Contrastive learning
Hàm mất mát	Multiple Negatives Ranking Loss
Mẫu âm	In-batch negatives + hard negatives (BM25/dense top-k sai)
Hạng r (nếu dùng LoRA)	16
Tốc độ học	1×10^{-5} đến 2×10^{-5}
Số epoch	3–5
Độ dài tối đa	512 token
Chỉ dẫn (instruction)	Chỉ áp dụng cho truy vấn
Đánh giá	Recall@k, MRR (phải re-embed toàn bộ corpus)

Bảng 4.6: Cấu hình fine-tuning reranker đề xuất.

Tham số	**Giá trị đề xuất**
Mô hình nền	Qwen3-Reranker (cross-encoder)
Đầu vào	Cặp (truy vấn, chunk)
Nhãn	Liên quan / không liên quan theo đáp án tham chiếu
Hàm mất mát	Cross-entropy (phân loại) hoặc margin/ranking loss
Mẫu âm	Hard negatives từ top-k sai
Hạng r (nếu dùng LoRA)	16
Tốc độ học	1×10^{-5} đến 2×10^{-5}
Số epoch	2–4
Đánh giá	Recall@5, MRR, nDCG (trước/sau)

Quy trình chung: làm sạch dữ liệu → định dạng theo loại mô hình → chia tập theo câu hỏi → huấn luyện → đánh giá trên tập kiểm định → đóng gói để phục vụ bằng vLLM. Lưu ý riêng cho embedding: sau khi thay mô hình phải *re-embed toàn bộ corpus* và dựng lại chỉ mục Qdrant trước khi đo lại Recall@k.

4.5 Triển khai multi-agent

4.5.1 Agent điều phối

Agent điều phối (host) tiếp nhận yêu cầu, duy trì ngữ cảnh hội thoại và quyết định định tuyến. Prompt của host quy định danh sách agent được phép gọi, tiêu chí định tuyến và cơ chế hợp nhất kết quả. Host cũng hỗ trợ tạo ticket khi cần chuyển sang hỗ trợ thủ công, với phân loại ticket theo nhóm nghiệp vụ (tư vấn, hỗ trợ ứng dụng, cước phí, đối soát thanh toán, hỗ trợ kỹ thuật).

4.5.2 Các agent chuyên biệt và công cụ

Các agent chuyên biệt giao tiếp qua A2A và truy cập công cụ qua MCP. Ví dụ, agent tra cứu bưu gửi cung cấp công cụ tra cứu theo SOAP và theo API nội bộ, cùng công cụ phân tích điểm nghẽn (bottleneck). Agent tra cứu bưu cục truy vấn cơ sở dữ liệu PostgreSQL có tìm kiếm mờ (fuzzy search) để xử

lý sai chính tả. Agent RAG sản phẩm – dịch vụ dùng Qdrant và reranker như mô tả.

4.5.3 Quy tắc định tuyến

Với miền EMS, hệ thống sử dụng một bộ khoảng 26 quy tắc chuyển agent, được nhóm theo các chủ đề: yêu cầu trực tiếp, khiếu nại/bồi thường, phản hồi, tác vụ vận hành, nghiệp vụ/hợp đồng, khách hàng VIP, an ninh/pháp lý, độ tin cậy thấp và chuyển cấp. Các quy tắc được lưu dưới dạng JSON có thể đọc bằng máy, bao gồm thông tin cần thu thập và chính sách theo giờ hành chính, giúp việc định tuyến nhất quán và dễ bảo trì.

4.6 Hệ thống quan sát và quản lý prompt

Mọi agent đều phát vết theo chuẩn OpenTelemetry tới một bộ thu (collector), sau đó phân phối tới Grafana Tempo và Langfuse. Vết ghi lại luồng gọi công cụ, mô hình được dùng, lượng token, thời gian tới token đầu tiên và latency tổng. Thông tin phiên và người dùng được truyền qua baggage để liên kết các vết trong cùng một hội thoại.

Prompt được quản lý tập trung qua MLflow Prompt Registry. Mỗi agent tham chiếu một prompt theo tên và nhãn (ví dụ `@production`), nhờ đó có thể cập nhật prompt cho tất cả agent mà không cần sửa mã hay triển khai lại. Cơ chế này tách cấu hình khỏi logic và hỗ trợ kiểm soát thay đổi.

4.7 Khó khăn và giải pháp

Kết chương

Chương này đã mô tả môi trường, dữ liệu, cấu hình RAG, thiết kế fine-tuning, triển khai multi-agent và hệ thống quan sát. Chương tiếp theo trình bày kết quả thực nghiệm và đánh giá.

Bảng 4.7: Khó khăn gặp phải và giải pháp áp dụng.

Khó khăn	**Giải pháp**
Stemming tiếng Việt kém hiệu quả	Tắt stemming, loại bỏ từ dừng trong BM25
Bảng biểu bị cắt vụn khi chunking	Chunking nhận biết cấu trúc, bảo toàn bảng
Tài liệu scan chất lượng thấp	Tiền xử lý OCR bằng mô hình nhận dạng chuyên dụng
Reranker làm giảm recall top-k nhỏ	Phân tích lỗi, điều chỉnh k và ngưỡng
Chi phí GPU cho mô hình lớn	Quantization FP8, phục vụ theo lô với vLLM
Dữ liệu nhạy cảm	Chạy cục bộ, ẩn PII, RBAC theo vai trò
Prompt phân tán, khó kiểm soát	MLflow Prompt Registry tập trung

Chương 5

Kết quả và đánh giá

Chương này trình bày kết quả thực nghiệm của hệ thống. Mục 5.1 mô tả phương pháp đánh giá; mục 5.2 trình bày kết quả retrieval; mục 5.3 trình bày kết quả sinh văn bản; mục 5.4 so sánh các hướng tiếp cận; mục 5.5 trình bày hiệu năng hệ thống; mục 5.6 nêu case study định tính; mục 5.7 ước lượng hiệu quả kinh tế – xã hội; và mục 5.8 thảo luận.

5.1 Phương pháp đánh giá

Đề tài đánh giá hệ thống theo ba lớp: (i) đánh giá retrieval; (ii) đánh giá sinh văn bản; và (iii) đánh giá hiệu năng hệ thống.

Đánh giá retrieval được thực hiện bằng một quy trình tái lập chính xác đường đi sản xuất: với mỗi câu hỏi, hệ thống chạy sparse retrieval BM25 và dense retrieval, lấy hợp ứng viên, rerank, rồi so sánh tập kết quả với đáp án tham chiếu. Các thước đo gồm Recall@k cho từng nhánh, Recall sau hợp, Recall@k sau rerank và MRR. Quy trình còn phân tách lỗi thành hai loại: lỗi *RECALL* (chunk đúng không được truy xuất) và lỗi *RERANK* (chunk đúng có trong ứng viên nhưng bị rerank loại khỏi top-k). Cách phân tách này giúp định vị điểm nghẽn chính xác.

Đánh giá sinh văn bản sử dụng hai phương pháp. Thứ nhất, LLM-as-judge cho điểm câu trả lời theo thang 0–10 dựa trên đáp án tham chiếu. Thứ hai, bộ thước đo RAGAS gồm bốn thành phần: Faithfulness, Answer Relevancy, Context Precision và Context Recall. Bộ đánh giá RAGAS được cài đặt theo hai phiên bản: một bản chính thức dùng thư viện RAGAS và một bản tự cài đặt (“RAGAS-4”) để kiểm soát chi tiết từng thành phần, với mô hình đánh giá độc lập.

Đánh giá hiệu năng đo throughput và inference latency của vLLM ở hai chế độ: một yêu cầu đơn (single-stream) và lô lớn (batch).

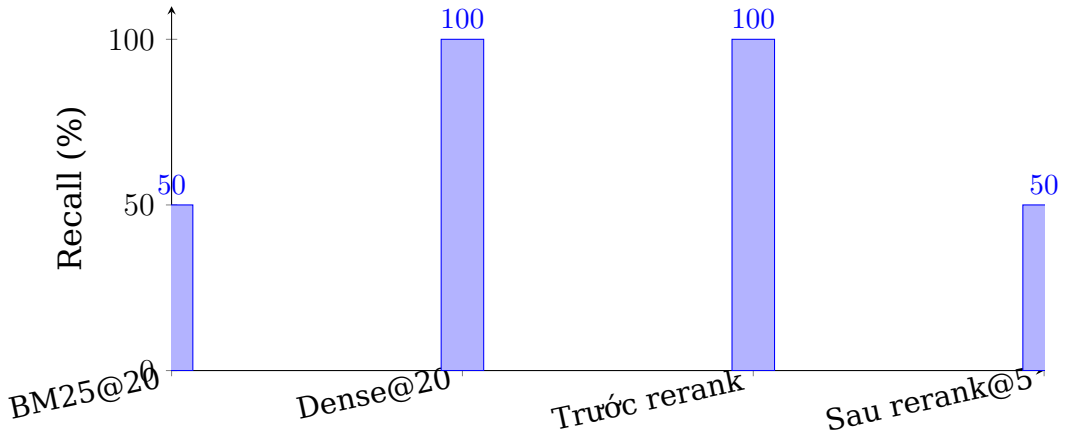
5.2 Kết quả retrieval

5.2.1 Kết quả tổng hợp

Bảng 5.1 trình bày kết quả retrieval trên bộ mẫu 6 câu hỏi có đáp án tham chiếu, với cấu hình 20/20/5 và ngưỡng điểm 0,0.

Bảng 5.1: Kết quả retrieval trên bộ mẫu (6 câu hỏi).

Thước đo	Giá trị	Tỉ lệ
Recall@20 (BM25)	3/6	50,0%
Recall@20 (dense)	6/6	100,0%
Recall trước rerank (BM25 ∪ dense)	6/6	100,0%
Recall@5 sau rerank (cấu hình sản xuất)	3/6	50,0%
MRR	0,3149	–



Hình 5.1: So sánh Recall giữa các cấu hình retrieval.

Kết quả cho thấy dense retrieval vượt trội hơn BM25 trên bộ mẫu này (100% so với 50%), phản ánh ưu thế của truy xuất ngữ nghĩa khi câu hỏi diễn đạt khác với từ khóa trong tài liệu. Hybrid retrieval đạt recall trước rerank là 100%, nghĩa là chunk đúng luôn nằm trong tập ứng viên. Tuy nhiên, sau rerank chỉ còn 50%, cho thấy bước rerank đã loại bỏ một phần chunk đúng.

5.2.2 Phân tích lỗi

Quy trình đánh giá phân tách lỗi và cho kết quả: lỗi *RERANK* chiếm 3/6 (50%), lỗi *RECALL* chiếm 0/6 (0%). Điều này khẳng định điểm nghẽn chính nằm ở khâu rerank chứ không phải khâu retrieval. Có hai nguyên nhân khả dĩ: (i) reranker có thể ưu tiên các chunk "trông giống" câu hỏi về mặt từ vựng nhưng thiếu thông tin đáp án; và (ii) khi số chunk sau rerank nhỏ ($k_2 = 5$), một sai sót xếp hạng có thể đẩy chunk đúng ra ngoài top-k. Hàm ý cải tiến gồm: tăng k_2, hiệu chỉnh ngưỡng điểm, hoặc kết hợp điểm rerank với điểm truy xuất ban đầu.

> *Bộ mẫu chỉ có 6 câu hỏi nên các tỉ lệ trên mang tính minh họa cho phương pháp và cần được kiểm chứng trên bộ lớn hơn (99 câu) trong các đợt đánh giá tiếp theo.*

5.3 Kết quả sinh văn bản và đánh giá agent

5.3.1 Đánh giá bởi LLM-as-judge

Trên bộ mẫu 3 câu hỏi, agent được đánh giá bởi mô hình ngôn ngữ theo thang 0–10, dựa trên so sánh với đáp án tham chiếu. Kết quả cho giá trị trung bình, trung vị, nhỏ nhất và lớn nhất đều bằng 10,00. Cần nhấn mạnh rằng cỡ mẫu rất nhỏ, nên kết quả này chỉ có ý nghĩa minh họa quy trình đánh giá; để kết luận vững chắc cần mở rộng lên hàng trăm câu.

5.3.2 Bộ đánh giá RAGAS trên bộ 99 câu hỏi EMS

Hệ thống đã xây dựng bộ đánh giá RAGAS cho miền EMS dựa trên bộ 99 câu hỏi – đáp án tham chiếu, với hai phiên bản cài đặt: một bản dùng thư viện RAGAS chính thức và một bản tự cài đặt ("RAGAS-4") để kiểm soát chi tiết từng thành phần, sử dụng mô hình đánh giá độc lập. Bốn thước đo được tính gồm Faithfulness, Answer Relevancy, Context Precision và Context Recall.

Hệ thống đã thu thập phản hồi cho toàn bộ 99 câu hỏi của bộ đánh giá: 99/99 câu thành công, hoàn tất trong 249,4 giây với 10 luồng xử lý song song. Mỗi bản ghi gồm câu hỏi, đáp án tham chiếu, câu trả lời của chatbot, ngữ cảnh retrieval và độ dài ký tự.

Bước tổng hợp điểm từ các phản hồi đã thu thập chưa hoàn tất tại thời điểm viết báo cáo, do đó báo cáo chưa công bố các giá trị RAGAS tổng hợp. Việc hoàn tất và công bố các điểm này là một nội dung của đợt đánh giá tiếp theo (xem mục 5.8 và chương 6).

5.4 So sánh RAG, fine-tuning và kiến trúc hybrid

Bảng 5.2 so sánh ba hướng tiếp cận dựa trên đặc điểm triển khai và kết quả quan sát được. Với RAG, các nhận định dựa trên kết quả thực nghiệm; với fine-tuning và kiến trúc hybrid, các nhận định mang tính phân tích thiết kế do chưa có kết quả huấn luyện thật.

Bảng 5.2: So sánh RAG, fine-tuning và kiến trúc hybrid trong bối cảnh VNPost.

Tiêu chí	**RAG**	**Fine-tuning**	**Lai**
Tri thức động	Rất tốt	Kém	Tốt
Trích nguồn	Có	Hạn chế	Có
Phong cách miền	Trung bình	Tốt	Tốt
Chi phí cập nhật	Thấp	Cao	Thấp–trung bình
Latency	Cao hơn	Thấp	Trung bình
Độ phức tạp	Trung bình	Cao	Cao
Đánh giá thực nghiệm	Có số liệu	Đề xuất	Đề xuất

Kết luận chính: RAG là lựa chọn phù hợp và khả thi nhất cho nhu cầu hiện tại của VNPost vì tri thức thay đổi thường xuyên và cần trích nguồn; fine-tuning bổ trợ ở lớp hành vi; và kiến trúc lai là đích đến dài hạn, khi dữ liệu hội thoại đã đủ chín để huấn luyện.

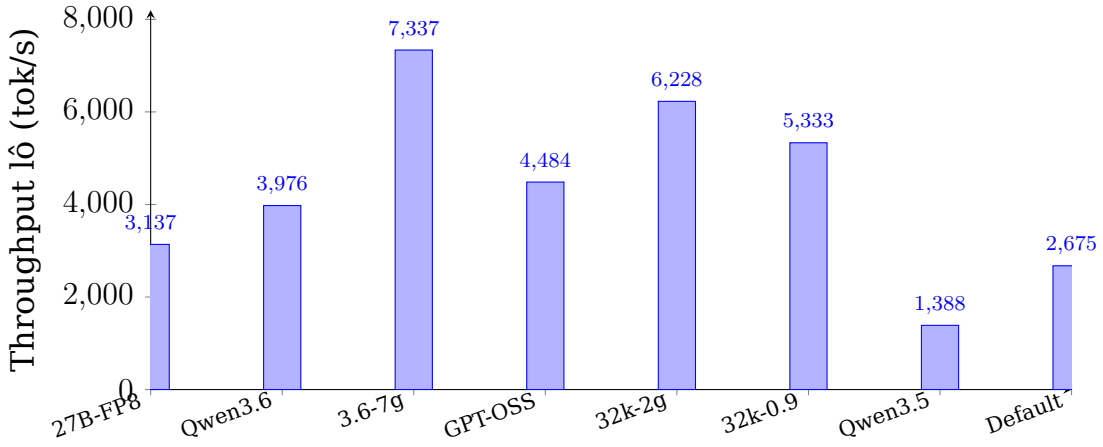
5.5 Hiệu năng hệ thống

Bảng 5.3 trình bày kết quả đo inference throughput của vLLM cho các mô hình khác nhau. Cột “đơn” là số token/giây cho một yêu cầu đơn; cột “lô” là throughput tối đa khi phục vụ đồng thời 1024 yêu cầu.

Kết quả cho thấy các mô hình cỡ 20–35 tỷ tham số với FP8 đạt throughput đơn khoảng 140–240 token/giây và throughput lô từ vài nghìn tới hơn 7.000

Bảng 5.3: Kết quả đo inference throughput vLLM.

Cấu hình mô hình	Đơn (tok/s)	Lô @1024 (tok/s)
Qwen3.8-27B-FP8	180,0	3.137
Qwen3.6 (K8s)	232,9	3.976
Qwen3.6 (K8s, 7g)	236,4	7.337
GPT-OSS-20B	242,8	4.484
Cấu hình 32k (2 GPU)	146,8	6.228
Cấu hình 32k (0,9)	138,0	5.333
Qwen3.5	50,1	1.388
Cấu hình mặc định	147,1	2.675



Hình 5.2: Throughput phục vụ lô của các cấu hình vLLM.

token/giây, đủ đáp ứng nhu cầu phục vụ đồng thời của một tổng công ty. Với latency mục tiêu vài giây cho câu trả lời ngắn, cấu hình hiện tại đáp ứng được yêu cầu trải nghiệm.

5.6 Case study định tính

Để minh họa hành vi thực tế, dưới đây là một số tình huống hội thoại ví dụ (không phải bản ghi hội thoại thật).

Tình huống 1 – Hỏi biểu phí. Người dùng hỏi: "Cước EMS đi Nhật cho 500g là bao nhiêu?". Agent điều phối định tuyến đến agent RAG sản phẩm – dịch vụ. Hệ thống lấy các chunk biểu phí EMS liên quan, rerank, rồi sinh câu trả lời kèm trích dẫn số hiệu quyết định. Nếu không tìm thấy ngữ cảnh đủ tin cậy, hệ thống thông báo thiếu thông tin thay vì đoán.

Tình huống 2 – Tra cứu bưu gửi. Người dùng cung cấp mã bưu gửi. Agent điều phối gọi agent tra cứu bưu gửi, agent này gọi công cụ tracking qua SOAP/API và trả về tình trạng mới nhất cùng các mốc thời gian.

Tình huống 3 – Tra cứu bưu cục. Người dùng hỏi địa chỉ bưu cục gần một địa điểm. Agent tra cứu bưu cục truy vấn cơ sở dữ liệu với tìm kiếm mờ để xử lý sai chính tả và trả về danh sách bưu cục phù hợp.

Tình huống 4 – Chuyển hỗ trợ thủ công. Khi yêu cầu vượt ngoài phạm vi hoặc cần xử lý nghiệp vụ phức tạp, agent điều phối tạo ticket và chuyển đến nhân viên, bảo đảm không bỏ sót yêu cầu.

5.7 Hiệu quả kinh tế – xã hội

Bên cạnh lợi ích về chất lượng phục vụ, hệ thống mang lại hiệu quả kinh tế đáng kể nhờ tiết kiệm thời gian tra cứu. Theo ước lượng từ một sáng kiến tự động hóa báo cáo và phân quyền đã được ghi nhận tại VNPost, giải pháp giúp tiết kiệm khoảng 30–60 phút mỗi ngày cho mỗi người dùng ở khâu tổng hợp báo cáo. Với giả định thận trọng rằng mỗi người dùng tiết kiệm tối thiểu 10 phút/ngày và quy mô khoảng 13.000 người dùng, mỗi ngày tiết kiệm được khoảng 2.167 giờ công. Quy đổi theo đơn giá 56.818 VNĐ/giờ,

22 ngày/tháng và 264 ngày/năm, giá trị tiết kiệm ước tính khoảng 32,5 tỷ VNĐ/năm.

Bảng 5.4: Ước lượng hiệu quả kinh tế của giải pháp.

Đại lượng	**Giá trị ước tính**
Thời gian tiết kiệm/người/ngày (giả định)	≥ 10 phút
Số người dùng	≈ 13.000
Giờ công tiết kiệm/ngày	≈ 2.167 giờ
Đơn giá giờ công	56.818 VNĐ/giờ
Số ngày làm việc	22 ngày/tháng, 264 ngày/năm
Giá trị tiết kiệm/năm	$\approx 32,5$ tỷ VNĐ

Đây là ước lượng dựa trên giả định và số liệu tham chiếu từ một sáng kiến liên quan; giá trị thực tế có thể khác tùy mức độ sử dụng và phạm vi triển khai.

5.8 Thảo luận

5.8.1 Ưu điểm

Hệ thống cho thấy RAG lai với rerank là giải pháp khả thi cho tri thức nội bộ động của VNPost: dense retrieval đạt recall rất cao, kiến trúc multi-agent giúp mở rộng theo nghiệp vụ, và việc phục vụ cục bộ đáp ứng yêu cầu bảo mật. Quan sát tập trung và quản lý prompt giúp vận hành và cải tiến liên tục.

5.8.2 Hạn chế

Thứ nhất, kết quả retrieval định lượng hiện dựa trên bộ mẫu nhỏ (6 câu), chưa đủ đại diện. Thứ hai, bước rerank làm giảm recall ở top-k nhỏ, cần cải tiến. Thứ ba, phần fine-tuning chưa được huấn luyện thật nên chưa có bằng chứng định lượng. Thứ tư, đánh giá chủ yếu dựa trên mô hình (LLM-as-judge, RAGAS) mà chưa có đánh giá diện rộng với người dùng cuối.

5.8.3 Tính khái quát

Kiến trúc và quy trình đánh giá đề xuất có thể áp dụng cho các tổ chức có đặc thù tương tự: khối tài liệu nội bộ lớn, nhu cầu tra cứu thường xuyên, và yêu cầu bảo mật dữ liệu. Việc thay thế kho tài liệu và agent chuyên biệt cho phép tái sử dụng nền tảng cho miền khác.

Kết chương

Chương này đã trình bày kết quả retrieval, kết quả sinh văn bản, so sánh các hướng tiếp cận, hiệu năng hệ thống, case study và hiệu quả kinh tế – xã hội. Chương tiếp theo kết luận và đề xuất hướng phát triển.

Chương 6

Kết luận và hướng phát triển

6.1 Kết luận

Báo cáo đã nghiên cứu và triển khai thực nghiệm một trợ lý ảo thông minh cho Tổng công ty Bưu điện Việt Nam, dựa trên mô hình ngôn ngữ lớn kết hợp kỹ thuật RAG (RAG) và fine-tuning (fine-tuning). Các kết quả chính bao gồm:

1. **Hệ thống hóa cơ sở lý thuyết:** làm rõ kiến trúc Transformer, cơ chế của RAG (chunking, embedding, sparse/dense/hybrid retrieval, rerank), các phương pháp fine-tuning (SFT, LoRA/QLoRA, DPO), các thước đo đánh giá và nền tảng triển khai (vLLM, FP8, MCP, A2A, OpenTelemetry, MLflow). Từ đó, báo cáo lập luận cho kiến trúc hybrid RAG + fine-tuning.
2. **Thiết kế kiến trúc multi-agent:** đề xuất kiến trúc gồm một agent điều phối và các agent chuyên biệt giao tiếp qua A2A, truy cập công cụ qua MCP, phục vụ cục bộ nhằm bảo đảm dữ liệu nội bộ không rời hạ tầng tổ chức.
3. **Triển khai pipeline RAG lai:** cài đặt sparse retrieval BM25 và dense retrieval trên Qdrant, kết hợp rerank bằng cross-encoder, cùng quy trình tiền xử lý tài liệu (OCR, Markdown hóa, metadata hóa, structure-aware chunking).
4. **Đề xuất pipeline fine-tuning:** thiết kế quy trình thu thập dữ liệu, ẩn PII và fine-tuning cho cả ba loại mô hình – LLM (LoRA/QLoRA), embedding model (contrastive, hard negatives) và reranker (cross-encoder) – rồi phục vụ bằng vLLM, kèm tiêu chí quyết định khi nào dùng RAG, fine-tuning hoặc kết hợp.
5. **Đánh giá định lượng:** trên bộ mẫu, dense retrieval đạt Recall@20 = 100% trong khi cấu hình sản xuất sau rerank đạt Recall@5 = 50% và MRR = 0,3149; phân tích lỗi cho thấy 50% lỗi thuộc khâu rerank và 0% thuộc khâu retrieval. Bộ đánh giá EMS gồm 99 câu đã được thu thập

phản hồi đầy đủ (99/99 trong 249,4 giây).

6. **Đánh giá hiệu năng và hiệu quả:** các mô hình cỡ 20–35 tỷ tham số với FP8 đạt throughput đơn 140–240 token/giây và throughput lô từ vài nghìn tới hơn 7.000 token/giây; ước tính tiết kiệm khoảng 32,5 tỷ VNĐ/năm dựa trên giả định thận trọng về thời gian tiết kiệm.

Tổng hợp lại, RAG là giải pháp phù hợp và khả thi cho nhu cầu tri thức động và trích nguồn của VNPost, fine-tuning bổ trợ ở lớp hành vi và phong cách nghiệp vụ, và kiến trúc hybrid là đích đến dài hạn mang lại giá trị thực tiễn cao.

6.2 Hạn chế

Bên cạnh kết quả đạt được, đề tài còn một số hạn chế:

- **Cỡ mẫu đánh giá retrieval nhỏ:** kết quả định lượng chi tiết dựa trên 6 câu hỏi, chưa đủ đại diện cho toàn miền nghiệp vụ. Bộ 99 câu đã được thu thập phản hồi nhưng chưa tổng hợp đầy đủ các thước đo.
- **Suy giảm recall ở khâu rerank:** cấu hình hiện tại làm mất một phần chunk đúng ở top-k nhỏ, cần cải tiến.
- **Chưa huấn luyện fine-tuning thật:** phần fine-tuning mới dừng ở thiết kế; chưa có bằng chứng định lượng về hiệu quả của mô hình fine-tuning.
- **Đánh giá chủ yếu dựa trên mô hình:** các thước đo RAGAS và LLM-as-judge có thể mang sai lệch của mô hình đánh giá; chưa có đánh giá diện rộng với người dùng cuối.
- **Phạm vi miền hẹp:** thực nghiệm tập trung vào bưu chính/EMS; tính khái quát sang các miền khác chưa được kiểm chứng.

6.3 Hướng phát triển

Trên cơ sở kết quả và hạn chế, đề tài đề xuất các hướng phát triển sau:

1. **Mở rộng và chuẩn hóa đánh giá:** hoàn tất tổng hợp điểm RAGAS trên bộ 99 câu và mở rộng lên hàng trăm tới hàng nghìn câu; bổ sung đánh giá A/B với người dùng cuối.
2. **Cải tiến retrieval và rerank:** điều chỉnh k_2 và ngưỡng điểm; thử nghiệm kết hợp điểm rerank với điểm retrieval; fine-tuning embed-

ding model và reranker trên dữ liệu tiếng Việt miền bưu chính để nâng Recall@5 sau rerank.

3. **Nâng cấp lên agentic RAG:** cho phép agent tự quyết định truy xuất nhiều vòng, cải viết truy vấn và gọi công cụ bổ trợ khi ngữ cảnh chưa đủ.
4. **Khai thác GraphRAG:** xây dựng đồ thị tri thức từ tài liệu để truy xuất theo quan hệ giữa các thực thể (dịch vụ, biểu phí, quy định, địa bàn), hỗ trợ câu hỏi phức tạp.
5. **Triển khai fine-tuning định kỳ:** khi đã tích lũy đủ log hội thoại chất lượng, tiến hành SFT/LoRA và đánh giá so với RAG thuần; xây dựng quy trình cập nhật định kỳ.
6. **Continual learning và cá nhân hóa:** cập nhật tri thức liên tục và học sở thích người dùng, đồng thời bảo đảm kiểm soát chất lượng và an toàn.
7. **Mở rộng đa ngôn ngữ và đa kênh:** hỗ trợ thêm ngôn ngữ và tích hợp vào các kênh như ứng dụng di động, website, tổng đài.
8. **Tăng cường an toàn và tuân thủ:** kiểm soát nội dung đầu ra, chống rò rỉ dữ liệu nhạy cảm, và bảo đảm tuân thủ quy định về dữ liệu cá nhân.

Kết chương

Chương này đã tóm tắt kết quả và đóng góp của đề tài, chỉ rõ các hạn chế cần khắc phục và đề xuất lộ trình phát triển. Những kết quả này là cơ sở để VNPost tiếp tục mở rộng hệ thống trợ lý ảo trong tương lai.

Phụ lục A

Prompt hệ thống mẫu

Phụ lục này trình bày các prompt hệ thống mẫu (đã lược giản) dùng cho agent điều phối và agent RAG. Prompt thực tế được quản lý tập trung qua MLflow Prompt Registry.

A.1. Prompt agent điều phối (host)

```
ạ
Bn là ợtr lý ảo ủca ôĨng công ty ưBu đệin ệVit Nam (VNPost).ệ
Nhim ụv: ê�runderiu yêu âcu ườngi dùng, duy trì ững ảcnh ộhi ạthoi, và định
   ê´
tuyn đến agent chuyên ệbit phù ợhp.

Các agent được phép ọgi:
- "ợTr lý tra ứcu ảsn âảhm ịdch ụv": câu ỏhi êv ê�runderbiu phí, quy định,
  chính sách ôbi ườthng, ờthi gian toàn trình.
- "ợTr lý định ịv ưBu ửải": tra ứcu tình ạtrng ưbu ửải theo mã.
- "ợTr lý tra ứcu ưbu ựcc": tra ứcu thông tinđị/a ỉch ưbu ựcc.

Nguyên ắtc:
1. ỉCh ảtr ờli ựda trên thông tin ừt agent chuyên ệbit ặhoc tài ệliu.
2. êNu êthiu thông tin, thông báo rõ và đề`ịngh ạto ticket ôñ ợtr.
3. Không ịba ôś ệliu, ôś ệhiu ăvn ảbn ặhoc chính sách.
4. Khi ậnhn êkt ảqu ừt agent chuyên ệbit, ợhp âńht thành câu ảtr ờli
   ạmch ạlc, có trích âdn ôngun khi có.
```

A.2. Prompt agent RAG

```
ạ
Bn là agent ỏhi đáp trên tài ệliu ộni ộb ủca VNPost.ạ
```

```
Bn được cung âćp:
- Câu ỏhi ủca ườngi dùng.
- Các chunk ững ảcnh truy âx́ut (kèm tiêu đề, ôś ệhiu, đường âđn ệtp).
- ịLch ưả ộhi ạthoi âgn đây.

Nguyên ăt́c ảtr ờli:
1. ỉCh ưả ụdng thông tin trong ững ảcnh retrieval. Không dùng êḱin
   úthc
   bên ngoài đểsuy êđin các con ôś ệnghip ụv.
2. êN̂u ững ảcnh không đủ đểảtr ờli, ảtr ờli đúng âm̃u:
   "Tôi ưcha tìm ât́hy thông tin này trong tài ệliu ộni ộb."
3. Trích âđn ônggun (tiêu đề/ôś ệhiu ăvn ảbn) cho ôm̃i thông tin quan
   ọtrng.
4. Trình bày ăńgn ọgn, đúng ọtrng tâm, dùng định ạdng phù ợhp (ảbng
   êńu
   là êb̉iu phí).
```

Phụ lục B

Bảng cấu hình tham số

Bảng B.1: Cấu hình tham số tổng hợp của hệ thống.

Tham số	**Giá trị**
Số chiều embedding	4096
Độ dài chunk tối đa	≈ 1000 ký tự
Độ chồng lấn chunk	bảo toàn ngữ cảnh liền kề
Số ứng viên dense (k_1^{dense})	20
Số ứng viên BM25 (k_1^{bm25})	10–20
Số chunk sau rerank (k_2)	5–10
Ngưỡng điểm rerank	0,0
Stemming tiếng Việt	tắt
Danh sách từ dừng	loại bỏ
Nhiệt độ sinh (temperature)	0,7
Top-p	0,80
Top-k (sampling)	20

Phụ lục C

Danh mục mô hình và thành phần

Bảng C.1: Các mô hình và thành phần chính được sử dụng.

Thành phần	Mô hình / công nghệ
Mô hình sinh ngôn ngữ	`vietnampost-llm` (gateway); họ Qwen 27B–35B (FP8)
Mô hình sinh thay thế	GPT-OSS-20B (FP8)
Mô hình embedding	Qwen3-Embedding-8B (4096 chiều)
Mô hình rerank	Qwen3-Reranker-8B (cross-encoder)
Mô hình OCR	olmocr-2-7b / lightonocr2-1b
Vector database	Qdrant v1.16.1 (dense COSINE + sparse IDF)
Cơ sở dữ liệu nghiệp vụ	PostgreSQL
Phục vụ inference	vLLM với PagedAttention và FP8
Quản lý prompt	MLflow Prompt Registry (`@production`)
Quan sát	OpenTelemetry, Langfuse, Grafana Tempo
Điều phối	Kubernetes, Docker Compose, Airflow

Phụ lục D

Mẫu hội thoại minh họa

```
[ờNgi dùng] ướCc EMS đi ậNht cho ưbu ửi 500g là bao nhiêu?

[Agent RAG] Theo êỉiu phí EMS ệhin hành (trích êQuyt định ôś ...),
   ướcc
cho ưbu ửi 500g đi ậNht ảBn ộthuc ậbc ... ớvi ứmc ướcc ... VND.ứ
Mc ướcc có êẻh thay đôỉ theo ờthi đêỉm và chính sách ưu đãi; vui lòng
   ê᾽
kim tra ạli ướtrc khi ửi.

[ờNgi dùng] ưBu ửi ủca tôi mã EG123456789VN đang ở đâu?

[Agent định ịv] ưBu ửi EG123456789VN đã đến ưbu ựcc phát ... vào
... . ạTrng thái ệhin ạti: đang phát.

[ờNgi dùng] âG̃n 12 êÑguyn Trãi, Hà ộNi có ưbu ựcc nào?

[Agent tra ứcu ưbu ựcc] Các ưbu ựcc âgn khu ựvc ... :
1. ưBu ựcc ... - địa ỉch ... - ờgi ởm ửca ...
2. ưBu ựcc ... - địa ỉch ... - ờgi ởm ửca ...
```

Các ví dụ trong phụ lục đã được lược giản và thay thế số liệu bằng dấu "..." để tránh công bố thông tin nội bộ. Mẫu hội thoại nhằm minh họa luồng định tuyến và định dạng câu trả lời, không phải dữ liệu thật.

Tài liệu tham khảo

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[2] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016.

[3] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.

[4] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.

[5] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:27730–27744, 2022.

[6] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems (NeurIPS)*, 36, 2023.

[7] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:24824–24837, 2022.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[9] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[10] Qdrant. Qdrant vector database documentation. `https://qdrant.tech/documentation/`, 2024. Truy cập 2026.

[11] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[12] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.

[13] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

[14] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[15] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[16] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the*

Association for Computational Linguistics (EACL): System Demonstrations, pages 150–158, 2024.

[17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.

[18] vLLM Project. vllm: Easy, fast, and cheap llm serving for everyone. `https://docs.vllm.ai`, 2024. Truy cập 2026.

[19] Anthropic. Model context protocol specification. `https://modelcontextprotocol.io`, 2024. Truy cập 2026.

[20] Google. Agent2agent (a2a) protocol. `https://a2a-protocol.org`, 2024. Truy cập 2026.

[21] OpenTelemetry. Opentelemetry documentation. `https://opentelemetry.io`, 2024. Truy cập 2026.

[22] Langfuse. Langfuse: Open source llm engineering platform. `https://langfuse.com`, 2024. Truy cập 2026.

[23] MLflow. Mlflow: A platform for the machine learning lifecycle. `https://mlflow.org`, 2024. Truy cập 2026.